



ECOLE MAROCAINE DES
SCIENCES DE L'INGENIEUR
Membre de
HONORIS UNITED UNIVERSITIES

FINAL-YEAR PROJECT REPORT

Program : Computer and Network Engineering (IIR), 5th year

Development of a Web Application for Construction Project Management and Planning

Submitted by :

Othmane BENHRADDI

Company Supervisor :

M. Mohamed BENABDELLAH-CHAOUNI

Academic Supervisor :

Pr. Mohammed EL ASSAD

Jury Members :

***M. EI ASSAD And M. OULAD HAJ THAMI
And M. ZAKRANI***

Host Organization :



CIVCO BTP

CIVCO BUILDING & ENGINEERING

Dédicaces

À ma grand-mère Fatima,

C'est toi qui m'as élevé, toi qui m'as appris à me tenir debout, à respecter et à donner sans rien attendre en retour. Les valeurs que tu m'as transmises éclairent chacun de mes choix.

Je prie Dieu chaque jour de te rendre la santé. Ce travail est, avant tout, le tien.

À mes parents, Mhamed et Malika,

Les mots me manquent pour dire l'étendue de ma reconnaissance. Par vos sacrifices, votre dévouement et votre amour sans condition, vous m'avez donné les moyens d'avancer et le courage de croire en mes rêves. Dans mes heures les plus sombres, votre présence a toujours été ma force la plus sûre. Ce diplôme est le vôtre autant que le mien.

À mes amis,

Vous avez été bien plus que des camarades de parcours. Vous avez porté mes doutes sans jamais les alourdir, fêté mes victoires sans jamais les diminuer, et vous êtes restés à mes côtés quand tout vacillait. Ces années n'auraient pas eu le même goût sans vous.

À mes professeurs,

Je vous dois bien plus que des connaissances. Votre exigence, parfois rude à vivre sur l'instant, s'est révélée être le plus précieux des cadeaux. Vous avez contribué, souvent sans même le savoir, à former le professionnel que j'aspire à devenir.

Remerciements

الحق لا يضادُّ الحقَّ، بل يوافقُه ويشهدُ له
Ibn Rushd (Averroès), Faṣl al-Maqāl

Je tiens à exprimer ma profonde gratitude envers toutes les personnes qui ont contribué à la réussite de ce projet et de mon stage au sein de **CIVCO BTP**.

Je suis profondément reconnaissant envers mon encadrant pédagogique, **Pr. Mohammed El Assad**, pour l'accompagnement bienveillant tout au long de ce projet, pour ses précieux conseils lors de nos réunions de suivi, et pour la structure claire qu'il m'a transmise pour la rédaction de ce rapport.

Je tiens à remercier chaleureusement mon encadrant professionnel, **M. Mohamed BENABDELLAH-CHAOUNI**, pour son implication directe au sein de **CIVCO BTP**, sa disponibilité constante, et pour m'avoir guidé tout au long du développement de cette application web de gestion et de planification des projets BTP. Son expertise du secteur et sa confiance m'ont permis de mener ce travail dans des conditions exigeantes et enrichissantes.

Je remercie également l'ensemble de l'équipe de **CIVCO BTP** pour m'avoir accueilli et intégré dans leur environnement professionnel, ainsi que pour la confiance accordée dans la réalisation de ce projet.

Enfin, je tiens à remercier l'**EMSI Casablanca** et l'ensemble des professeurs qui m'ont préparé à ce défi au cours de mes années d'études rigoureuses.

Résumé

Ce projet de fin d'études, réalisé au sein de l'entreprise **CIVCO BTP**, porte sur la conception et le développement d'une application web dédiée à la gestion et à la planification des projets de construction de l'entreprise. Face à la multiplication des chantiers et à la diversité des intervenants, CIVCO BTP souhaitait disposer d'un outil numérique centralisé, capable de structurer l'activité quotidienne et de faciliter le partage d'information entre les services.

L'application couvre l'ensemble du cycle de vie d'un projet : création et suivi des chantiers par phases, gestion des clients et de leurs contacts, affectation des équipes, planification des tâches, établissement des devis et des factures, production des bons de livraison, suivi des contrats et consultation d'un tableau de bord synthétique. Elle vise à remplacer les outils dispersés (tableurs, documents papier, échanges informels) par une plateforme unique, accessible aux différents profils métier de l'entreprise selon leurs droits d'accès.

La solution repose sur une architecture *full-stack* : un *backend* API développé avec **Laravel** et **PHP**, exposant des services **REST** sécurisés et s'appuyant sur une base de données relationnelle, couplé à une interface utilisateur **React** moderne organisée en **SPA** (*Single Page Application*). Un système de rôles et de permissions (**RBAC**) garantit que chaque utilisateur accède uniquement aux fonctionnalités correspondant à son profil.

Ce rapport présente le contexte du projet, les choix technologiques retenus, la conception fonctionnelle et technique du système, ainsi que la réalisation et la validation des principaux modules développés. Il s'articule autour de quatre chapitres : cadrage et méthodologie, état de l'art, conception **UML** du système, puis implémentation détaillée des fonctionnalités livrées.

Parmi les modules opérationnels livrés figurent la gestion des projets par phases avec cartographie **Leaflet**, la production et le suivi des devis, factures et bons de livraison, le portail client pour la consultation et la validation des documents, la messagerie interne, la planification des tâches, le suivi des contrats et avenants, ainsi qu'un module de configuration des rôles et des droits d'accès. Un journal d'activité assure par ailleurs la traçabilité des opérations sensibles au sein de l'application.

Mots-clés : BTP, Gestion de projets, Application web, ERP, Laravel, React, Construction, Devis, Facturation, Planification.

Abstract

This end-of-studies project, carried out at **CIVCO BTP**, focuses on the design and development of a web application dedicated to managing and planning the company's construction projects. Faced with a growing number of projects and stakeholders, CIVCO BTP sought a centralized digital tool capable of structuring daily operations and improving information sharing across departments.

The application covers the full project lifecycle : creating and tracking construction sites by phase, managing clients and contacts, assigning teams, planning tasks, issuing quotes and invoices, producing delivery notes, monitoring contracts, and viewing a consolidated dashboard. It aims to replace scattered tools (spreadsheets, paper documents, informal communication) with a single platform accessible to the company's various business roles according to their access rights.

The solution follows a full-stack architecture : a **Laravel** and **PHP** API *backend* exposing secure **REST** services backed by a relational database, paired with a modern **React** user interface organized as a **SPA** (*Single Page Application*). A role-based access control system (**RBAC**) ensures that each user can access only the features relevant to their profile.

This report presents the project context, the technology choices, the functional and technical system design, and the implementation and validation of the main modules developed. It is organized into four chapters : project framing and methodology, state of the art, **UML** system design, and detailed implementation of the delivered features.

Among the operational modules delivered are phase-based project management with **Leaflet** mapping, production and tracking of quotes, invoices and delivery notes, a client portal for document consultation and validation, internal messaging, task planning, contract and amendment tracking, and a role and permissions configuration module. An activity log also ensures traceability of sensitive operations within the application.

Keywords : Construction, Project Management, Web Application, ERP, Laravel, React, Building Industry, Quotes, Invoicing, Planning.

ملخص

يتناول هذا المشروع الختامي، المنجز داخل شركة BTP CIVCO، تصميم وتطوير تطبيق ويب مخصص لإدارة وتخطيط مشاريع البناء الخاصة بالشركة. وبسبب تزايد عدد الورش وتعدد الأطراف المعنية، سعت BTP CIVCO إلى امتلاك أداة رقمية مركزية تُنظّم النشاط اليومي وتُسهّل تبادل المعلومات بين مختلف الأقسام.

يغطي التطبيق دورة حياة المشروع بالكامل : إنشاء الورش ومتابعتها حسب المراحل، إدارة الزبناء وجهات الاتصال، تعيين الفرق، تخطيط المهام، إعداد العروض والفواتير، إنتاج بونات التسليم، متابعة العقود، والاطلاع على لوحة معلومات موحدة. يهدف إلى استبدال الأدوات المتفرقة (جداول البيانات، الوثائق الورقية، التواصل غير الرسمي) بمنصة واحدة يمكن للأدوار المهنية المختلفة في الشركة الوصول إليها وفق صلاحياتهم.

تعتمد الحل على بنية *full-stack* : خادم API مطوّر بـ **Laravel** و **PHP** يُوفّر خدمات **REST** مؤمنة ويعتمد على قاعدة بيانات علائقية، مقترناً بواجهة مستخدم حديثة بـ **React** منظمة كـ **SPA**. يضمن نظام الأدوار والصلاحيات (**RBAC**) أن يصل كل مستخدم فقط إلى الوظائف المرتبطة بملفه.

يقدم هذا التقرير سياق المشروع، والخيارات التقنية، والتصميم الوظيفي والتقني للنظام، إضافة إلى تنفيذ الوحدات الرئيسية والتحقق منها. وينقسم إلى أربعة فصول : الإطار العام والمنهجية، حالة الفن، التصميم **UML** للنظام، ثم تنفيذ الوظائف المسلّمة.

تشمل الوحدات التشغيلية المسلّمة إدارة المشاريع حسب المراحل مع خرائط **Leaflet**، إنتاج ومتابعة العروض والفواتير وبونات التسليم، بوابة الزبناء لاستشارة الوثائق والتحقق منها، المراسلة الداخلية، تخطيط المهام، متابعة العقود والملحقات، إضافة إلى وحدة ضبط الأدوار والصلاحيات. كما يضمن سجل النشاط تتبع العمليات الحساسة داخل التطبيق.

الكلمات المفتاحية : البناء، إدارة المشاريع، تطبيق ويب، **ERP**، **Laravel**، **React** قطاع البناء، العروض، الفوترة، التخطيط.

Table des matières

Dédicaces	i
Remerciements	ii
Résumé	iii
Abstract	iv
ملخص	v
Liste des figures	xii
Liste des tableaux	xiii
Liste des Abréviations	xiv
Introduction Générale	1
Chapitre 1 :Cadrage du Projet	2
1.1 Introduction	2
1.2 Présentation de l'organisme d'accueil	2
1.2.1 Historique et identité	2
1.2.2 Organisation de l'entreprise	3
1.2.3 Domaines d'activité	4
1.2.4 Moyens humains et matériels	5
1.3 Analyse de l'existant	5
1.3.1 Processus métier actuels	5
1.3.2 Cartographie des outils utilisés	6
1.3.3 Besoins exprimés par les services	6
1.4 Contexte et problématique	7
1.4.1 Constats et limites des pratiques actuelles	7
1.4.2 Problématique	7
1.5 Périmètre du projet	8
1.5.1 Fonctionnalités incluses	8
1.5.2 Limites et hors périmètre	8
1.6 Objectifs du projet	9
1.6.1 Objectif général	9

1.6.2	Objectifs fonctionnels	9
1.6.3	Objectifs techniques	9
1.7	Méthodologie adoptée	10
1.7.1	Démarche de développement	10
1.7.2	Gestion de projet avec Trello	10
1.7.3	Suivi académique et professionnel	11
1.7.4	Outils et environnement de travail	11
1.8	Planification du stage	12
1.8.1	Planning détaillé par période	12
Chapitre 2 :État de l'Art et Étude Théorique		14
2.1	Introduction	14
2.2	Les applications web et l'architecture full-stack	14
2.2.1	Évolution des applications web	14
2.2.2	Architecture client-serveur et tiers	15
2.2.3	Single Page Application (SPA)	15
2.2.4	Panorama des stacks full-stack	16
2.2.5	Échange de données et format JSON	16
2.2.6	Patterns CRUD et ressources métier	16
2.3	Le framework Laravel et l'écosystème PHP	16
2.3.1	Présentation de PHP et Laravel	16
2.3.2	Architecture MVC dans Laravel	17
2.3.3	Composants clés de l'écosystème Laravel	17
2.3.4	Comparaison avec les alternatives PHP	18
2.3.5	Laravel dans le projet CIVCO BTP	18
2.3.6	Middleware, Resources et couche de présentation API	18
2.4	React et les Single Page Applications	19
2.4.1	Définition et principes des SPA	19
2.4.2	React : bibliothèque déclarative	19
2.4.3	Écosystème React du projet	20
2.4.4	Communication avec le backend	20
2.4.5	Comparaison React / Vue / Angular	21
2.4.6	Organisation du code frontend	21
2.4.7	Interface utilisateur et Tailwind CSS	21
2.4.8	Internationalisation	21
2.5	Les bases de données relationnelles et l'ORM	22
2.5.1	Modèle relationnel et SGBD	22
2.5.2	L'ORM et Eloquent	22
2.5.3	Normalisation et intégrité des données	22

2.5.4	Modèle de données métier	23
2.6	Les API REST et l'authentification	23
2.6.1	Principe des API REST	23
2.6.2	Authentification et contrôle d'accès	24
2.7	ERP et digitalisation du secteur BTP	24
2.7.1	Enjeux de la gestion intégrée en BTP	24
2.7.2	Panorama des solutions existantes	24
2.7.3	Digitalisation des BET au Maroc	25
2.8	Références comparatives (Benchmarks) et justification des choix	26
2.8.1	Comparaison des frameworks backend	26
2.8.2	Comparaison des frameworks frontend	26
2.8.3	Comparaison des SGBD	27
2.8.4	Synthèse des choix	27
2.9	Méthodologies de développement logiciel	27
2.9.1	Approches traditionnelles et agiles	27
2.9.2	Application au projet CIVCO BTP	28
2.9.3	Qualité logicielle et bonnes pratiques	28
Chapitre 3 : Conception du Système		30
3.1	Introduction	30
3.2	Analyse des besoins	30
3.2.1	Identification des acteurs	30
3.2.2	Cas d'utilisation	31
3.2.3	Exigences fonctionnelles	32
3.2.4	Exigences non-fonctionnelles	33
3.2.5	Scénarios d'utilisation détaillés	33
3.3	Architecture du système	34
3.3.1	Architecture globale en trois couches	34
3.3.2	Organisation du backend Laravel	35
3.3.3	Organisation du frontend React	35
3.3.4	Protocoles de communication	35
3.3.5	Flux d'authentification	36
3.3.6	Matrice des permissions par profil	36
3.4	Conception des modules métier	37
3.4.1	Module Projets	37
3.4.2	Module Clients	37
3.4.3	Module Documents commerciaux	38
3.4.4	Module Contrats	38
3.4.5	Module Tâches	38

3.4.6	Module Tableau de bord et carte	39
3.4.7	Module Portail client et messagerie	39
3.4.8	Module Configuration et historique	40
3.5	Modélisation de la base de données	40
3.5.1	Principes de conception	40
3.5.2	Tables principales	41
3.5.3	Relations et contraintes d'intégrité	41
3.6	Conception de l'interface utilisateur	42
3.6.1	Principes ergonomiques	42
3.6.2	Charte graphique et navigation	42
3.6.3	Parcours utilisateurs types	43
3.7	Conception de la sécurité et de la traçabilité	43
3.7.1	Modèle de menaces et contre-mesures	43
3.7.2	Journalisation des actions	43
3.7.3	Gestion des impressions officielles	44
3.8	Modélisation UML	44
3.8.1	Diagramme de cas d'utilisation	44
3.8.2	Diagramme de séquence	45
3.8.3	Diagramme de classes	46
3.8.4	Diagramme d'activité	48
Chapitre 4 :Réalisation et Implémentation		50
4.1	Introduction	50
4.2	Environnement technique	50
4.2.1	Stack technologique	50
4.2.2	Outils de développement	51
4.2.3	Architecture de déploiement	51
4.3	Implémentation du backend	55
4.3.1	Structure du projet	55
4.3.2	API REST : endpoints principaux	55
4.3.3	Services métier	56
4.3.4	Authentification et contrôle d'accès	57
4.3.5	Migrations et modèle de données	57
4.3.6	Middleware et sécurité des routes	57
4.3.7	Exemple : création d'un devis	58
4.3.8	Modules complémentaires	58
4.4	Implémentation du frontend	59
4.4.1	Architecture React	59
4.4.2	Client HTTP et authentification	60

4.4.3	Routage et contrôle d'accès	61
4.4.4	Composants transverses	61
4.4.5	Internationalisation	61
4.4.6	Détail des modules frontend	61
4.4.7	Gestion d'état et hooks	62
4.5	Déroulement du développement et livrables	62
4.5.1	Phases de réalisation	62
4.5.2	Synthèse quantitative des livrables	63
4.5.3	Bonnes pratiques adoptées	63
4.6	Interfaces principales	64
4.6.1	Page de connexion	64
4.6.2	Tableau de bord	65
4.6.3	Gestion des projets	66
4.6.4	Gestion des clients	67
4.6.5	Documents commerciaux	68
4.6.6	Planification des tâches	69
4.6.7	Carte des chantiers	70
4.6.8	Portail client	71
4.6.9	Configuration	72
4.7	Tests et validation	73
4.7.1	Tests automatisés backend	73
4.7.2	Campagne de tests manuels	73
4.7.3	Validation qualitative	74
4.7.4	Synthèse des tests	74
4.8	Difficultés rencontrées et solutions	75
4.8.1	Gestion des permissions granulaires	75
4.8.2	Impression documentaire	75
4.8.3	Géolocalisation des chantiers	75
4.8.4	Synchronisation frontend / backend	75
Conclusion Générale et Perspectives		77
Bibliographie et Webographie		81
Chapitre A : Annexes		83
A.1	Extraits de code source	83
A.1.1	Authentification API (AuthController.php)	83
A.1.2	Conversion devis vers facture (QuoteService.php)	84
A.1.3	Modèle Projet (Project.php)	85
A.1.4	Client HTTP frontend (client.js)	86

A.2	Configuration : variables d'environnement	87
A.3	Comptes de démonstration	88
A.4	Référentiel des endpoints API	88
A.5	Schéma des entités principales	89
A.6	Guide de démarrage rapide	90

Table des figures

1.1	Logo de CIVCO BTP — CivCo Building & Engineering	2
1.2	Organisation fonctionnelle de CIVCO BTP	4
2.1	Architecture three-tier de l'application CIVCO BTP	15
2.2	Flux de communication React → Laravel → SGBD	20
3.1	Diagramme de cas d'utilisation de l'application CIVCO BTP	45
3.2	Diagramme de séquence : création et enregistrement d'un devis	46
3.3	Diagramme de classes simplifié du modèle de données métier	47
3.4	Diagramme d'activité : cycle de vie d'un projet de construction	48
4.1	Page de connexion : authentification par email et mot de passe	64
4.2	Tableau de bord : vue synthétique de l'activité (projets, tâches, indicateurs)	65
4.3	Module Projets : liste des chantiers et fiche détail (phases, équipe, documents)	66
4.4	Module Clients : fiches clients, contacts et archivage	67
4.5	Module Devis : création avec lignes de détail et calcul automatique des totaux	68
4.6	Module Factures : suivi des factures et état de paiement	68
4.7	Module Tâches : planification, assignation et suivi par projet	69
4.8	Carte des projets : visualisation géographique des chantiers (Leaflet)	70
4.9	Portail client : consultation des projets, acceptation de devis et signature . .	71
4.10	Module Configuration : rôles, modèles de documents, contrôles d'impression	72

Liste des tableaux

1.1	Outils actuels et limites identifiées	6
1.2	Phases principales du projet de stage	12
1.3	Planning détaillé du stage (avril – juillet 2026)	12
2.1	Panorama des stacks full-stack pour applications métier	16
2.2	Comparaison des frameworks PHP pour une API REST	18
2.3	Composants de l'écosystème frontend	20
2.4	Comparaison des principaux frameworks frontend (2025-2026)	21
2.5	Entités principales du modèle de données	23
2.6	Panorama de solutions de gestion applicables au BTP	25
2.7	Comparaison des frameworks backend pour une API métier	26
2.8	Comparaison des frameworks frontend	26
2.9	Comparaison des systèmes de gestion de bases de données	27
2.10	Synthèse des choix technologiques et critères décisifs	27
3.1	Matrice des permissions par profil utilisateur (synthèse)	36
3.2	Tables principales de la base de données	41
3.3	Menaces identifiées et contre-mesures associées	43
3.4	Description des classes et de leurs relations	47
4.1	Versions des composants de la stack technique	51
4.2	Comparatif d'hébergement : OVHcloud vs o2switch	53
4.3	Variables d'environnement principales du fichier .env	54
4.4	Phases de développement et modules livrés	63
4.5	Synthèse de la couverture de tests par module	74
A.1	Variables d'environnement principales	87
A.2	Comptes de démonstration (mot de passe : password)	88
A.3	Principaux endpoints de l'API REST	89
A.4	Attributs principaux des entités métier	89

Liste des Abréviations

API	Application Programming Interface	JSON	JavaScript Object Notation
BDD	Base de Données	Kanban	Méthode de gestion visuelle des tâches (à faire, en cours, terminé)
BET	Bureau d'Études Techniques	KPI	Key Performance Indicator
BL	Bon de Livraison	MVC	Model-View-Controller
BTP	Bâtiment et Travaux Publics	ORM	Object-Relational Mapping
CRUD	Create, Read, Update, Delete	PDF	Portable Document Format
CRM	Customer Relationship Management	PFE	Projet de Fin d'Études
CSRF	Cross-Site Request Forgery	RBAC	Role-Based Access Control
DOM	Document Object Model	REST	Representational State Transfer
EMSI	École Marocaine des Sciences de l'Ingénieur	SGBD	Système de Gestion de Bases de Données
ERP	Enterprise Resource Planning	SPA	Single Page Application
HTML	HyperText Markup Language	SQL	Structured Query Language
HTTP	HyperText Transfer Protocol	UI	User Interface
HTTPS	HyperText Transfer Protocol Secure	UML	Unified Modeling Language
IIR	Ingénierie Informatique et Réseaux	UX	User Experience

Introduction Générale

Le secteur du **BTP** (*Bâtiment et Travaux Publics*) exige une coordination rigoureuse entre acteurs, documents et échéances. Chaque chantier mobilise des équipes pluridisciplinaires, génère des devis, contrats, factures et bons de livraison, et nécessite un suivi continu de l'avancement. Pourtant, de nombreuses entreprises s'appuient encore sur des outils dispersés — tableurs, documents papier, échanges informels — qui fragmentent l'information, ralentissent la prise de décision et augmentent le risque d'erreurs. La digitalisation, notamment via un **ERP** (*Enterprise Resource Planning*) adapté au BTP, permet de centraliser la gestion des projets, des clients et des documents commerciaux au sein d'une plateforme unique.

Ce projet de fin d'études, réalisé au sein de **CIVCO BTP**, consiste à concevoir et développer une **application web** de gestion et de planification des projets de construction de l'entreprise. Cette solution vise à accompagner les équipes internes dans le pilotage quotidien de leurs chantiers : suivi des projets et des tâches, gestion des clients, production des documents commerciaux, suivi des contrats et consultation d'un tableau de bord de synthèse. Elle répond ainsi au besoin de fiabiliser les processus métier et d'améliorer la collaboration entre les différents services.

La solution repose sur une architecture *full-stack* : un *backend* API **Laravel / PHP** exposant des services **REST** sécurisés, couplé à une interface **React** en **SPA** (*Single Page Application*), avec un contrôle d'accès par rôles (**RBAC**) adapté aux profils de l'entreprise.

Ce rapport est organisé en quatre chapitres :

- **Chapitre 1** : présentation de CIVCO BTP, analyse de la problématique, objectifs du projet et méthodologie adoptée.
- **Chapitre 2** : état de l'art et justification des choix technologiques (*full-stack*, Laravel, React, ERP BTP).
- **Chapitre 3** : conception du système (besoins fonctionnels, acteurs, architecture, modélisation **UML**).
- **Chapitre 4** : réalisation des modules principaux et validation de l'application par des tests.

Chapitre 1 : Cadrage du Projet

1.1 Introduction

Ce chapitre présente le contexte dans lequel ce projet a été réalisé. Il introduit **CIVCO BTP** (*CivCo Building & Engineering*) en tant qu'organisme d'accueil, décrit la problématique métier qui a motivé le projet, et définit les objectifs ainsi que la méthodologie adoptés tout au long du stage.

1.2 Présentation de l'organisme d'accueil

1.2.1 Historique et identité

CivCo Building & Engineering est un **BET** (*Bureau d'Études Techniques*) marocain, basé à **Casablanca**, spécialisé dans le génie civil, le bâtiment, l'ingénierie des infrastructures et les solutions informatiques au service de la gestion et du suivi des projets (Figure 1.1).



Figure 1.1 : Logo de CIVCO BTP — CivCo Building & Engineering

L'entreprise a été créée pour accompagner les maîtres d'ouvrage, les architectes et les entreprises de construction dans la conception, l'étude et le suivi technique de leurs projets. Au fil de son développement, elle a constitué un pôle informatique chargé de la transformation numérique interne et du développement d'applications répondant aux besoins des différents services.

Grâce à son expertise et au respect des normes nationales et internationales, CIVCO BTP intervient sur des projets de tailles variées : bâtiments résidentiels, ouvrages industriels et

infrastructures publiques. Sa politique repose sur l'innovation, la qualité, la sécurité des ouvrages et la satisfaction client, en intégrant les technologies de l'ingénierie et de l'informatique.

1.2.2 Organisation de l'entreprise

L'organisation de CIVCO BTP repose sur une structure fonctionnelle favorisant la collaboration entre les services techniques, administratifs et informatiques. Les principaux services sont les suivants :

- **Direction Générale** : définition de la stratégie, supervision des activités et développement commercial.
- **Service Études Techniques** : études de conception, calculs de structures et production des plans techniques.
- **Service Suivi des Travaux** : contrôle de l'exécution, respect des délais et suivi des chantiers.
- **Service Administration et Finance** : ressources humaines, comptabilité, achats et gestion administrative.
- **Service Qualité et Sécurité** : contrôle qualité, respect des normes et gestion de la sécurité.
- **Service Informatique, Réseaux et Développement** : développement d'applications internes et web, administration des réseaux et serveurs, gestion des bases de données, support utilisateurs, cybersécurité et digitalisation des processus.

La Figure 1.2 présente la répartition fonctionnelle des services au sein de l'entreprise.

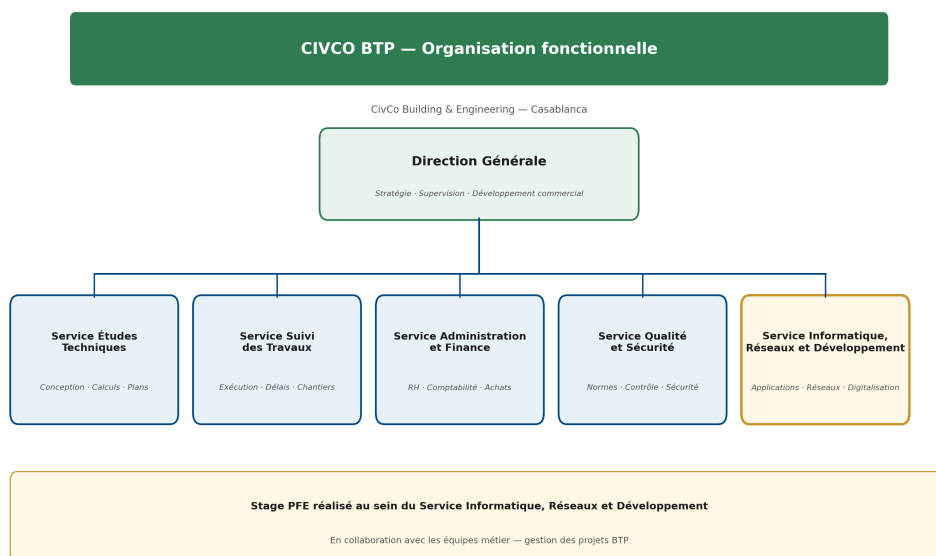


Figure 1.2 : Organisation fonctionnelle de CIVCO BTP

Ce stage s’est déroulé au sein du **Service Informatique, Réseaux et Développement**, en étroite collaboration avec les équipes métier concernées par la gestion des projets.

1.2.3 Domaines d’activité

Les activités de CIVCO BTP couvrent plusieurs domaines complémentaires :

Génie civil et études de structures. Études de bâtiments résidentiels et industriels, ouvrages publics et infrastructures ; calcul en béton armé, structures métalliques, dimensionnement des fondations et vérification de stabilité.

Assistance technique et conseil. Assistance à maîtrise d’ouvrage, contrôle des travaux, coordination des intervenants, réception des ouvrages, expertise technique, études de faisabilité et optimisation des solutions constructives.

Informatique et développement. Développement d’applications web et de logiciels métiers, conception et administration de bases de données, maintenance applicative, sécurisation des systèmes d’information et déploiement de solutions numériques au service de la productivité et du suivi des projets.

Exemples de projets réalisés. CIVCO BTP intervient sur des chantiers de natures variées, illustrant la diversité de son portefeuille :

- **Bâtiments résidentiels** : immeubles collectifs, villas haut standing, lotissements avec voirie et réseaux.
- **Ouvrages industriels** : halls de production, entrepôts logistiques, extensions d’usines existantes.

- **Infrastructures publiques** : voiries urbaines, ouvrages d’assainissement, aménagements d’espaces publics.
- **Rénovation et réhabilitation** : mise aux normes parasismiques, extension de bâtiments patrimoniaux, restructuration de locaux tertiaires.

Cette diversité impose à l’entreprise une capacité d’adaptation et un suivi rigoureux de chaque affaire, renforçant le besoin d’un outil de gestion centralisé.

1.2.4 Moyens humains et matériels

Moyens humains. L’entreprise s’appuie sur une équipe pluridisciplinaire : direction générale, ingénieurs génie civil et structures, dessinateurs-projeteurs, conducteurs de travaux, techniciens, développeurs informatiques, administrateurs réseaux et systèmes, personnel administratif et financier. Cette diversité de compétences permet de répondre aux exigences techniques et organisationnelles des projets confiés à CIVCO BTP.

Moyens matériels. Sur le plan technique, l’entreprise dispose d’un parc informatique (postes de travail, serveurs, équipements réseau), de logiciels de conception (AutoCAD, Revit, Robot Structural Analysis), d’outils de développement (Visual Studio Code, Git, GitHub) et de systèmes de gestion de bases de données (MySQL, PostgreSQL, SQL Server). Des véhicules de chantier, une connectivité haut débit et des plateformes collaboratives complètent l’environnement de travail.

1.3 Analyse de l’existant

Avant de définir la solution cible, une phase d’observation et d’entretiens a été menée auprès des différents services de CIVCO BTP. L’objectif était de comprendre comment l’information circule aujourd’hui, quels sont les points de friction et quels gains une application intégrée pourrait apporter.

1.3.1 Processus métier actuels

Le cycle de vie d’un projet de construction chez CIVCO BTP peut se décomposer en plusieurs étapes successives, impliquant des acteurs distincts :

1. **Prospection et relation client** : identification du besoin, échanges préliminaires, constitution de la fiche client (coordonnées, contacts, historique).
2. **Étude et chiffrage** : analyse technique, estimation des coûts, production d’un devis détaillé avec lignes de prestations.

3. **Contractualisation** : rédaction du contrat, éventuels avenants en cas de modification du périmètre ou du budget.
4. **Planification** : découpage en phases, affectation de l'équipe, définition des tâches opérationnelles et des jalons.
5. **Exécution** : suivi de chantier, mise à jour de l'avancement, production des bons de livraison et des documents de suivi.
6. **Facturation et clôture** : émission des factures, suivi des paiements, archivage du dossier projet.

Chaque étape génère des documents et des données qui doivent être partagés entre le service commercial, les ingénieurs, les conducteurs de travaux, la comptabilité et parfois le client final. Aujourd'hui, cette circulation repose en grande partie sur des échanges manuels.

1.3.2 Cartographie des outils utilisés

Le Tableau 1.1 synthétise les outils actuellement mobilisés par service et leurs limites respectives.

Tableau 1.1 : Outils actuels et limites identifiées

Service	Outils	Limites observées
Commercial	Excel, Word, courriel	Devis non centralisés, ressaisies
Études / Projets	AutoCAD, Revit, fichiers partagés	Pas de lien avec le suivi financier
Chantier	WhatsApp, tableurs	Informations éphémères, peu traçables
Comptabilité	Logiciel comptable, PDF	Saisie manuelle des factures
Direction	Rapports ponctuels	Absence de tableau de bord temps réel

Cette cartographie a confirmé l'absence d'un **référentiel unique** reliant clients, projets, documents et équipes. Chaque service dispose de sa propre organisation documentaire, ce qui complique la consolidation des indicateurs et la reprise d'information lors d'un changement d'interlocuteur.

1.3.3 Besoins exprimés par les services

Les entretiens menés avec l'encadrant professionnel et les responsables de service ont permis de formaliser les attentes prioritaires :

- **Direction** : vision consolidée de l'activité (projets actifs, retards, charge des équipes).

- **Commercial** : création rapide de devis, historique client, conversion simplifiée en facture.
- **Ingénieurs / chefs de projet** : suivi des phases, des tâches et de l'équipe affectée à chaque chantier.
- **Comptabilité** : accès aux factures, suivi des paiements et état de facturation par projet.
- **Clients** : consultation de l'avancement et validation des documents sans dépendre des échanges par courriel.

Ces besoins ont orienté le périmètre fonctionnel de l'application développée.

1.4 Contexte et problématique

1.4.1 Constats et limites des pratiques actuelles

Au quotidien, la gestion des projets de construction chez CIVCO BTP mobilise plusieurs services : études techniques, suivi de travaux, administration, finance et qualité. Chaque chantier génère un volume important d'informations — clients, phases, tâches, devis, factures, contrats, bons de livraison — qui doivent être partagées entre les intervenants.

Or, une partie de ces processus repose encore sur des outils non intégrés : fichiers Excel, documents Word ou PDF isolés, échanges par courriel ou messagerie instantanée. Cette dispersion entraîne plusieurs difficultés :

- **Fragmentation de l'information** : les données d'un même projet sont éparpillées entre plusieurs fichiers et supports.
- **Manque de visibilité** : difficulté à disposer d'une vue d'ensemble sur l'avancement des chantiers et des tâches en cours.
- **Risque d'erreurs** : ressaisies manuelles, versions multiples de documents et absence de source unique de vérité.
- **Lenteur de coordination** : dépendance aux échanges informels entre services pour obtenir une information à jour.

1.4.2 Problématique

Face à l'augmentation du nombre de projets et à la complexité croissante de leur suivi, CIVCO BTP a identifié le besoin de disposer d'un **outil numérique centralisé** dédié à la gestion et à la planification de ses projets de construction.

La problématique de ce projet peut se formuler ainsi :

Comment concevoir et développer une application web permettant à CIVCO BTP de centraliser, structurer et piloter l'ensemble du cycle de vie de ses projets de construction, tout en s'adaptant aux différents profils métier de l'entreprise ?

1.5 Périmètre du projet

1.5.1 Fonctionnalités incluses

Le périmètre retenu pour ce stage couvre les modules suivants, considérés comme prioritaires pour une première version exploitable en interne :

- Authentification sécurisée et gestion des rôles (**RBAC**).
- Gestion des projets (phases, équipe, avancement, géolocalisation).
- Gestion des clients et provisionnement du portail externe.
- Production des documents commerciaux (devis, factures, bons de livraison).
- Suivi contractuel (contrats, avenants, modèles de documents).
- Planification et suivi des tâches opérationnelles.
- Tableau de bord, carte des chantiers et historique des actions.
- Messagerie projet entre équipe interne et client.

1.5.2 Limites et hors périmètre

Certains sujets ont été volontairement exclus ou reportés à une phase ultérieure, afin de concentrer l'effort de développement sur le cœur métier :

- **Comptabilité générale** : l'application gère les factures et paiements projet, mais ne remplace pas le logiciel comptable de l'entreprise.
- **CAO / BIM** : les outils de conception (AutoCAD, Revit) restent indépendants ; seules les métadonnées projet sont centralisées.
- **Gestion des stocks** : non couverte dans cette version.
- **Application mobile native** : l'interface responsive web est privilégiée ; une application dédiée est envisagée en perspective.
- **Interopérabilité ERP tiers** : pas d'intégration avec des solutions externes (SAP, Sage, etc.) dans le cadre de ce stage.

Cette délimitation garantit un livrable cohérent et testable dans la durée du stage, tout en laissant une base évolutive pour les extensions futures.

1.6 Objectifs du projet

1.6.1 Objectif général

Concevoir et développer une **application web** de gestion et de planification des projets de construction pour CIVCO BTP, capable de couvrir les principaux processus métier liés au suivi des chantiers et à la production des documents commerciaux.

1.6.2 Objectifs fonctionnels

- **Gestion des projets** : créer, consulter et suivre les chantiers par phases, avec affectation des équipes et suivi de l'avancement.
- **Gestion des clients** : centraliser les informations clients, contacts et historique des projets associés.
- **Documents commerciaux** : produire et suivre les devis, factures et bons de livraison (BL) liés aux projets.
- **Suivi contractuel** : gérer les contrats et les avenants associés aux chantiers.
- **Planification des tâches** : organiser le travail opérationnel par phases et attribuer les tâches aux collaborateurs.
- **Tableau de bord** : offrir une vue synthétique de l'activité (projets en cours, indicateurs clés, tâches prioritaires).
- **Contrôle d'accès** : garantir que chaque utilisateur accède uniquement aux fonctionnalités correspondant à son rôle (RBAC).

1.6.3 Objectifs techniques

- Mettre en place une architecture *full-stack* avec un *backend* API **Laravel / PHP** et une interface **React** en **SPA**.
- Concevoir une base de données relationnelle (**SQL**) modélisant les entités métier (projets, clients, documents, utilisateurs, rôles).
- Exposer des services **REST** sécurisés pour la communication entre le *frontend* et le *backend*.
- Assurer une interface ergonomique, responsive et adaptée à un usage professionnel quotidien.
- Valider le bon fonctionnement des modules développés par des tests manuels et, le cas échéant, des tests automatisés.

1.7 Méthodologie adoptée

1.7.1 Démarche de développement

Le projet a été mené selon une approche **itérative et incrémentale**, adaptée au contexte d'un stage en entreprise. Le travail s'est déroulé en plusieurs étapes :

1. **Analyse des besoins** : recueil des attentes auprès de l'encadrant professionnel et identification des modules prioritaires.
2. **Conception** : modélisation des entités métier, définition de l'architecture technique et des cas d'utilisation principaux.
3. **Développement par modules** : implémentation progressive des fonctionnalités (projets, clients, documents commerciaux, tâches, tableau de bord, etc.).
4. **Tests et validation** : vérification du comportement de chaque module et correction des anomalies identifiées.
5. **Documentation** : rédaction du présent rapport et documentation technique du code produit.

Chaque itération visait à livrer un incrément fonctionnel, démontrable lors des réunions de suivi, avant de passer au module suivant.

1.7.2 Gestion de projet avec Trello

Le suivi opérationnel du stage a été organisé à l'aide de **Trello**, outil de gestion de projet en ligne basé sur la méthode **Kanban**. Un tableau dédié au projet regroupait les tâches sous forme de cartes réparties en colonnes selon leur état : *À faire*, *En cours*, *En revue* et *Terminé*.

Chaque carte correspondait à une fonctionnalité, une correction ou une tâche de conception (par exemple : « Module devis — API backend », « Écran tableau de bord », « Tests RoleAccess »). Les cartes précisaient une brève description, une priorité et, le cas échéant, une échéance. Cette organisation a permis de :

- visualiser l'avancement global du projet en un coup d'œil ;
- prioriser les modules à développer en accord avec l'encadrant professionnel ;
- conserver une trace des tâches réalisées et de celles restant à traiter ;
- préparer les démonstrations lors des réunions de suivi hebdomadaires.

Trello a ainsi complété Git/GitHub : le premier pour la planification et le suivi des livrables métier, le second pour le versionnement du code source.

1.7.3 Suivi académique et professionnel

Suivi académique (EMSI Casablanca) :

- Encadrant pédagogique : **Pr. Mohammed El Assad**
- Réunions régulières de suivi, validation de la démarche et préparation de la soutenance finale.

Suivi professionnel (CIVCO BTP) :

- Encadrant professionnel : **M. Mohamed BENABDELLAH-CHAOUNI**
- Réunions de suivi pour présenter l'avancement, valider les choix fonctionnels et recueillir les retours métier.
- Alignement continu du développement avec les besoins réels de l'entreprise.

1.7.4 Outils et environnement de travail

Outils de développement :

- **Trello** : gestion de projet et suivi des tâches (tableau Kanban).
- **Git / GitHub** : gestion de versions et suivi des évolutions du code.
- **Visual Studio Code** : environnement de développement principal.
- **Laravel** : framework *backend* (**MVC**, **ORM** Eloquent).
- **React + Vite** : interface utilisateur (**SPA**).
- **MySQL / PostgreSQL** : base de données relationnelle.

Environnements :

- **Développement** : poste local avec serveur Laravel et serveur de développement Vite (*hot-reload*).
- **Tests** : validation manuelle des parcours utilisateurs et tests fonctionnels sur les modules livrés.

1.8 Planification du stage

Le stage s'est déroulé sur la période **avril – juillet 2026**, au sein du service informatique de CIVCO BTP. La planification générale du projet peut se résumer comme suit :

Tableau 1.2 : Phases principales du projet de stage

Phase	Activités principales
Analyse	Étude de l'existant, recueil des besoins, définition du périmètre
Conception	Modélisation UML, architecture technique, maquettage des interfaces
Développement	Implémentation progressive des modules (<i>backend</i> et <i>frontend</i>)
Tests	Vérification fonctionnelle, corrections et stabilisation
Rédaction	Rédaction du rapport de fin d'études et préparation de la soutenance

1.8.1 Planning détaillé par période

Le Tableau 1.3 détaille la répartition des activités sur les quatre mois du stage. Chaque période correspond à un jalon de démonstration validé avec l'encadrant professionnel.

Tableau 1.3 : Planning détaillé du stage (avril – juillet 2026)

Période	Livrables et activités
Avril 2026	Analyse de l'existant, entretiens métier, rédaction du cahier des charges fonctionnel, validation du périmètre avec l'encadrant professionnel
Mai 2026	Conception UML (cas d'utilisation, séquence, classes), modélisation de la base de données, architecture technique three-tier, maquettage des écrans principaux
Juin 2026	Développement backend Laravel (authentification, projets, clients, devis, factures), mise en place du frontend React (navigation, tableau de bord), premières démonstrations des modules livrés
Juillet 2026	Portail client, tâches, configuration des rôles, tests PHPUnit et validation manuelle, stabilisation, rédaction du rapport de fin d'études et préparation de la soutenance

Cette planification incrémentale a permis de livrer des modules testables au fur et à mesure, tout en ajustant les priorités en fonction des retours terrain. Par exemple, le module d'impression documentaire et le suivi des copies officielles ont été intégrés en cours de développement suite à une demande exprimée par la direction administrative.

Conclusion

Ce chapitre a posé le cadre du projet de fin d'études réalisé au sein de **CIVCO BTP**. Nous avons présenté l'organisme d'accueil, son organisation et les domaines d'activité du bureau d'études, afin de situer le projet dans son contexte métier BTP. L'analyse de l'existant a mis en évidence la dispersion des outils (tableurs, documents isolés, échanges informels) et la difficulté à disposer d'une vision consolidée des chantiers, des clients et des documents commerciaux.

À partir de cette problématique, nous avons formulé les objectifs fonctionnels et techniques de l'application : centraliser la gestion des projets, des clients, des devis, des factures et du suivi opérationnel, tout en garantissant un contrôle d'accès par rôles adapté aux différents profils de l'entreprise. Le périmètre retenu couvre les modules prioritaires identifiés avec l'encadrant professionnel, dans une logique de livrable progressif et testable.

La méthodologie adoptée combine une approche **agile incrémentale**, un suivi **Kanban** via **Trello** et un versionnement **Git/GitHub**. Le planning sur quatre mois (avril – juillet 2026) a structuré les phases d'analyse, de conception, de développement et de validation, avec des jalons de démonstration réguliers. Ce cadrage constitue la base sur laquelle s'appuient les chapitres suivants.

Le chapitre suivant expose l'état de l'art des technologies mobilisées et justifie les choix techniques retenus — architecture *full-stack*, **Laravel**, **React** et base de données relationnelle — pour la réalisation de l'application.

Chapitre 2 : État de l’Art et Étude Théorique

2.1 Introduction

L’application développée pour CIVCO BTP s’appuie sur un ensemble de technologies web modernes, réparties entre un *backend* API, une base de données relationnelle et une interface utilisateur riche. Ce chapitre constitue le socle théorique du rapport : il présente les concepts et les technologies étudiés, puis justifie les choix retenus pour la réalisation du projet.

Nous abordons successivement les **architectures des applications web**, le **framework Laravel** côté serveur, les **interfaces React en SPA**, les **bases de données relationnelles et l’ORM**, les **API REST et l’authentification**, ainsi que les **solutions ERP et outils de gestion de projets dans le secteur du BTP**. Le chapitre se conclut par une référence comparative (*benchmark*) justifiant chaque choix technologique retenu.

2.2 Les applications web et l’architecture full-stack

2.2.1 Évolution des applications web

Les applications de gestion d’entreprise ont connu une évolution majeure au cours des deux dernières décennies. Les premières solutions reposaient sur des architectures **monolithiques** : logique métier, interface et base de données regroupées dans un même programme, souvent accessible via un navigateur avec rechargement complet de page à chaque action.

L’émergence des **API** (*Application Programming Interface*) a séparé la couche de présentation de la couche métier. Le navigateur ou une application mobile consomme des services distants via **HTTP**, échangeant des données structurées en **JSON**. Cette découpe a ouvert la voie aux architectures *full-stack* modernes, dans lesquelles un *frontend* JavaScript riche dialogue avec un *backend* API indépendant [1].

2.2.2 Architecture client-serveur et tiers

L'architecture **three-tier** (*trois tiers*) structure l'application en trois couches distinctes :

1. **Couche présentation** : interface utilisateur (React), exécutée dans le navigateur.
2. **Couche métier** : logique applicative, règles de gestion, authentification (Laravel).
3. **Couche données** : persistance relationnelle (MySQL / PostgreSQL).

Cette séparation améliore la maintenabilité : chaque couche peut évoluer indépendamment tant que les contrats d'interface (API REST) sont respectés. Fowler [2] décrit ce découpage comme un patron fondamental des applications d'entreprise.

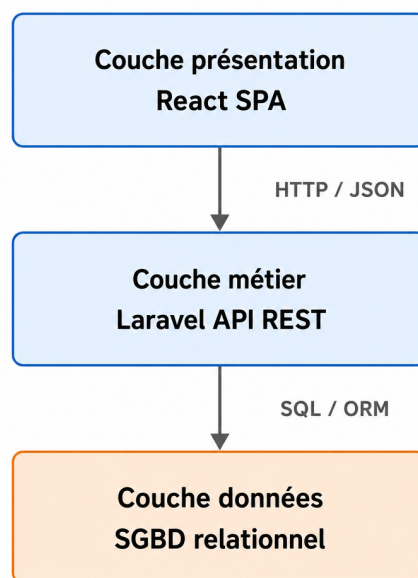


Figure 2.1 : Architecture three-tier de l'application CIVCO BTP

2.2.3 Single Page Application (SPA)

Une **SPA** (*Single Page Application*) charge une seule page HTML, puis met à jour dynamiquement le contenu via JavaScript sans rechargement complet. Les avantages pour une application métier sont significatifs :

- **Fluidité** : navigation instantanée entre les modules (projets, clients, factures).
- **Expérience utilisateur** : interfaces réactives, proches des applications de bureau.
- **Réutilisation de l'état** : conservation du contexte utilisateur (filtres, session) entre les écrans.

Le couplage d'une SPA React avec une API Laravel constitue aujourd'hui un standard industriel pour les tableaux de bord et outils de gestion internes.

2.2.4 Panorama des stacks full-stack

Tableau 2.1 : Panorama des stacks full-stack pour applications métier

Stack	Langages	Caractéristiques
LAMP / LEMP	PHP + MySQL	Mature, hébergement abordable, Laravel/Symfony
MERN / MEAN	JavaScript	Un seul langage, Node.js + MongoDB
Django + React	Python + JS	Rapidité backend, ORM Django
Laravel + React	PHP + JS	API REST + SPA, écosystème riche
.NET + Angular	C# + TS	Écosystème Microsoft, entreprise

Pour CIVCO BTP, la stack **Laravel + React** (Tableau 2.1) offre le meilleur compromis entre productivité, courbe d'apprentissage et adéquation avec les compétences visées en filière IIR : développement web, bases de données et architecture logicielle.

2.2.5 Échange de données et format JSON

Les API modernes échangent des données structurées en **JSON** (*JavaScript Object Notation*), format léger et lisible, natif en JavaScript et bien supporté par PHP/Laravel. Chaque ressource métier est sérialisée en objet JSON : un projet expose son identifiant, son client associé, ses phases, son statut et ses dates ; un devis expose ses lignes, montants et statut de validation.

Ce format facilite le découplage frontend/backend : React consomme directement les réponses JSON, les mappe vers ses composants et met à jour l'interface sans transformation complexe. Les codes de statut HTTP (200, 201, 422, 403) signalent le résultat de chaque opération de façon standardisée.

2.2.6 Patterns CRUD et ressources métier

La majorité des modules de l'application suivent le pattern **CRUD** (*Create, Read, Update, Delete*) : créer un client, lister les projets, modifier une facture, archiver un document. Laravel formalise ce pattern via les contrôleurs de ressources et les routes RESTful, ce qui garantit une cohérence d'API sur l'ensemble des modules (clients, projets, devis, factures, tâches, contrats).

2.3 Le framework Laravel et l'écosystème PHP

2.3.1 Présentation de PHP et Laravel

PHP est un langage de script côté serveur largement déployé sur le web. Historiquement associé au développement de sites dynamiques, il a évolué vers des versions modernes (8.x)

intégrant un typage strict, des performances accrues (JIT) et un écosystème mature de frameworks.

Laravel [3] est le framework PHP le plus adopté pour construire des applications web et des API. Créé par Taylor Otwell, il impose une structure claire autour du patron **MVC** (*Model-View-Controller*) et propose des composants intégrés pour les tâches courantes : routage, validation, authentification, files d'attente, envoi de courriels et gestion des fichiers.

2.3.2 Architecture MVC dans Laravel

Le patron **MVC** répartit les responsabilités :

- **Modèle** : représentation des entités métier et accès aux données (Eloquent).
- **Vue** : dans une API, les *Resources* et *Collections* formatent les réponses JSON pour le client.
- **Contrôleur** : reçoit la requête HTTP, applique la validation, invoque les services métier et retourne la réponse.

Cette organisation facilite la lecture du code et l'ajout de nouveaux modules. Pour l'application CIVCO BTP, chaque domaine métier (projets, clients, devis, factures) dispose de son contrôleur, de ses modèles et de ses règles de validation.

2.3.3 Composants clés de l'écosystème Laravel

Routage et validation

Le fichier de routes (`routes/api.php`) déclare les points d'entrée de l'API. Chaque route est associée à un contrôleur et protégée par des *middleware* (vérification d'authentification, contrôle de permissions, contexte entreprise).

Les **Form Requests** encapsulent les règles de validation des données entrantes (ex. : champs obligatoires d'un devis, formats de dates, montants positifs), garantissant l'intégrité des données avant persistance.

Services et logique métier

Au-delà des contrôleurs, la logique complexe est extraite dans des classes **Service** (`QuoteService`, `PrintTrackingService`, etc.). Ce découpage respecte le principe de responsabilité unique : le contrôleur orchestre, le service implémente la règle métier.

Migrations et seeders

Les **migrations** versionnent le schéma de la base de données sous forme de fichiers PHP exécutables de façon reproductible. Les **seeders** initialisent des données de référence (rôles, permissions, données de démonstration), indispensables pour tester l'application en conditions proches du réel.

2.3.4 Comparaison avec les alternatives PHP

Tableau 2.2 : Comparaison des frameworks PHP pour une API REST

Framework	Philosophie	Forces	Contexte idéal
Laravel	Convention over configuration	Écosystème complet, Sanctum, Eloquent	API + SPA, PME/ETI
Symfony	Composants réutilisables	Modularité, robustesse	Grandes applications
CodeIgniter	Minimaliste	Légèreté	Petits projets

Laravel ressort du Tableau 2.2 pour un projet de gestion métier nécessitant authentification, RBAC, documents et évolutivité — le profil exact du projet CIVCO BTP.

2.3.5 Laravel dans le projet CIVCO BTP

L'API Laravel expose plus de quarante contrôleurs organisés par domaine, des *middleware* de contrôle d'accès et un ensemble de services métier. Cette structure permet d'ajouter progressivement de nouveaux modules (avenants contractuels, modèles de documents, suivi d'impression) sans remettre en cause l'architecture existante.

2.3.6 Middleware, Resources et couche de présentation API

Middleware

Les **middleware** Laravel interceptent chaque requête HTTP avant qu'elle n'atteigne le contrôleur. Dans le projet, plusieurs middlewares assurent la sécurité et le contexte métier :

- **Authentification Sanctum** : vérifie que l'utilisateur est connecté.
- **Contrôle de permissions** : valide que l'utilisateur possède la permission requise (`project.view`, `quote.manage`, etc.).
- **Contexte entreprise** : identifie la société courante via l'en-tête `X-Company-Id`, garantissant l'isolation des données métier.

API Resources

Les classes **Resource** (`ProjectResource`, `InvoiceResource`, etc.) transforment les modèles Eloquent en tableaux JSON normalisés. Elles permettent de contrôler précisément les champs exposés à l'interface, d'inclure conditionnellement des relations (`client`, `lines`, `teamMembers`) et d'uniformiser le format de réponse sur toute l'API.

Validation et gestion des erreurs

Lorsqu'une requête échoue la validation, Laravel retourne une réponse 422 avec le détail des champs en erreur. Le frontend React intercepte ces réponses pour afficher des messages à l'utilisateur, améliorant l'expérience de saisie sur les formulaires de devis, factures et fiches projets.

2.4 React et les Single Page Applications

2.4.1 Définition et principes des SPA

Une **SPA** (*Single Page Application*) est une application web dont l'interface est construite dynamiquement en JavaScript. Contrairement aux sites classiques où chaque clic déclenche un rechargement HTML complet, la SPA charge le shell de l'application une fois, puis met à jour le DOM (*Document Object Model*) au fil des interactions utilisateur.

Ce modèle repose sur le **routing côté client** : la bibliothèque React Router associe chaque URL à un composant React (`/projects`, `/clients`, `/invoices`), sans solliciter le serveur pour le HTML. Seules les données nécessaires sont récupérées via l'API REST.

2.4.2 React : bibliothèque declarative

React [4], développé par Meta, est une bibliothèque JavaScript pour construire des interfaces utilisateur par **composants** réutilisables. Chaque composant encapsule structure (JSX), état et comportement.

Les principes fondamentaux de React sont :

- **Composants fonctionnels** : fonctions JavaScript retournant du JSX, composées hiérarchiquement (`Sidebar`, `Dashboard`, `ProjectDetailPage`, etc.).
- **État local** (`useState`) : données modifiables au sein d'un composant (formulaires, filtres, modales).
- **Effets de bord** (`useEffect`) : chargement de données API au montage du composant ou lors d'un changement de paramètre.

- **Context API** : partage d'état global (authentification, thème, langue) sans propager les props à travers toute l'arborescence.

Cette approche declarative simplifie la construction d'interfaces complexes : l'état de l'interface est une fonction de l'état applicatif.

2.4.3 Écosystème React du projet

L'application CIVCO BTP s'appuie sur plusieurs bibliothèques complémentaires :

Tableau 2.3 : Composants de l'écosystème frontend

Outil	Rôle	Usage dans le projet
Vite [5]	Build tool	Compilation rapide, HMR en développement
React Router	Routing	Navigation entre modules ERP
Axios	Client HTTP	Appels API REST avec intercepteurs
Tailwind CSS [6]	Styles utilitaires	Interface sombre cohérente
Recharts	Graphiques	Tableau de bord, KPIs
Leaflet	Cartographie	Carte des projets / chantiers

2.4.4 Communication avec le backend

Le *frontend* React consomme l'API Laravel via des modules dédiés (`api/projects.js`, `api/clients.js`, etc.). Un client Axios centralisé configure :

- l'envoi automatique des cookies de session (Sanctum) ;
- le jeton CSRF pour les requêtes POST, PUT et DELETE ;
- la gestion des erreurs 401 (redirection vers la page de connexion).

Ce patron *API-first* garantit que toute action utilisateur (créer un devis, archiver un client, assigner une tâche) transite par la couche métier Laravel, où les règles de validation et de permissions sont appliquées.

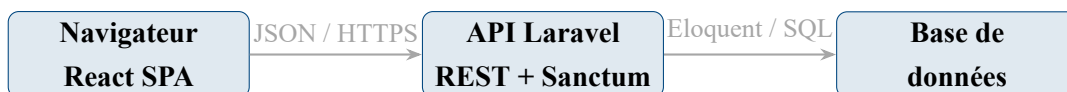


Figure 2.2 : Flux de communication React → Laravel → SGBD

2.4.5 Comparaison React / Vue / Angular

Tableau 2.4 : Comparaison des principaux frameworks frontend (2025-2026)

Critère	React	Vue.js	Angular
Flexibilité	Élevée (bibliothèque)	Élevée (progressif)	Opinionated (framework complet)
Courbe d'apprentissage	Modérée	Faible	Élevée
Marché de l'emploi	Très large	Large	Large (entreprise)
Écosystème SPA	React Router, Vite	Vue Router, Vite	Router intégré

React est retenu pour sa flexibilité, son adoption industrielle et la disponibilité de bibliothèques adaptées aux tableaux de bord métier (graphiques, cartes, composants modulaires), comme le montre le Tableau 2.4.

2.4.6 Organisation du code frontend

Le code React est structuré par **features** (features/projects/, features/clients/, features/invoices/), chacune regroupant pages, composants et utilitaires propres au module. Les composants transverses (Sidebar, PermissionGate, modales) vivent dans components/. Cette organisation par domaine facilite la navigation dans le code et l'ajout de nouvelles fonctionnalités sans impacter les modules existants.

2.4.7 Interface utilisateur et Tailwind CSS

L'interface adopte un design sombre professionnel, cohérent sur tous les modules. **Tailwind CSS** [6] est un framework CSS utilitaire : les classes (flex, rounded-lg, bg-gray-800) s'appliquent directement dans le JSX. Cette approche accélère le prototypage et garantit une cohérence visuelle sur les écrans de l'application (listes, fiches détail, modales, tableaux de bord).

Le **tableau de bord** illustre l'intérêt de React pour la visualisation métier : graphiques (Recharts), widgets configurables, indicateurs **KPI** et accès rapide aux projets en cours. La carte des chantiers (Leaflet) offre une vue géographique des projets, pertinente pour une entreprise disposant de plusieurs sites simultanés.

2.4.8 Internationalisation

L'application supporte le **français** (langue par défaut) et l'**anglais** via un contexte de langue et des fichiers de traduction (locales/fr.js, locales/en.js). Cette préparation facilite l'usage de l'outil par des collaborateurs anglophones ou une future extension à d'autres contextes linguistiques.

2.5 Les bases de données relationnelles et l'ORM

2.5.1 Modèle relationnel et SGBD

Une application de gestion de projets manipule des entités fortement structurées et liées entre elles : clients, projets, phases, tâches, devis, factures, contrats, utilisateurs et rôles. Le **modèle relationnel**, formalisé par Codd, organise ces données en tables reliées par des clés étrangères, garantissant l'intégrité référentielle et la cohérence des enregistrements [2].

Les **SGBD** (*Systèmes de Gestion de Bases de Données*) relationnels les plus répandus en environnement web sont **MySQL** [7] et **PostgreSQL** [8]. MySQL se distingue par sa simplicité de déploiement et sa large adoption ; PostgreSQL offre des fonctionnalités avancées (contraintes complexes, types JSON, performances sur de gros volumes). Pour une application métier comme celle de CIVCO BTP, les deux systèmes conviennent ; le choix final dépend de l'environnement d'hébergement et des besoins de montée en charge.

2.5.2 L'ORM et Eloquent

Manipuler directement le langage **SQL** dans chaque composant applicatif alourdit le code et complique la maintenance. L'**ORM** (*Object-Relational Mapping*) fait correspondre les tables de la base à des classes du langage de programmation. Dans Laravel, **Eloquent** fournit une couche d'abstraction élégante :

- chaque table est représentée par un **modèle** (Project, Client, Invoice, etc.) ;
- les relations (hasMany, belongsTo, belongsToMany) expriment les liens métier de façon déclarative ;
- les migrations versionnent le schéma de la base et facilitent le déploiement entre environnements ;
- les *seeders* permettent de peupler la base avec des données de démonstration.

Cette approche accélère le développement, réduit les erreurs de requêtes et aligne le code objet sur le modèle métier de l'entreprise.

2.5.3 Normalisation et intégrité des données

La conception d'une base pour un ERP léger repose sur plusieurs principes :

- **Normalisation** : éviter la duplication d'informations (un client est enregistré une seule fois, référencé par ses projets et documents).

- **Contraintes d'intégrité** : clés étrangères, unicité des identifiants métier (numéros de devis, de facture).
- **Traçabilité** : horodatage des enregistrements (`created_at`, `updated_at`) et journalisation des actions sensibles.

Ces principes sont essentiels dans un contexte BTP où les documents commerciaux et contractuels doivent rester cohérents et auditable.

2.5.4 Modèle de données métier

Le schéma relationnel de l'application modélise les entités principales du cycle de vie d'un projet CIVCO BTP :

Tableau 2.5 : Entités principales du modèle de données

Entité	Rôle
Client	Maître d'ouvrage ou client commercial, contacts associés
Project	Chantier / projet, lié à un client, phases et équipe
ProjectPhase	Découpage temporel du projet (études, exécution, réception)
Task	Tâche opérationnelle assignée à un collaborateur
Quote / Invoice	Documents commerciaux avec lignes de détail
Contract	Contrat et avenants liés au projet
User / Role	Utilisateurs internes et permissions RBAC

Les relations entre ces entités (un client possède plusieurs projets, un projet contient plusieurs phases et tâches, un devis peut être converti en facture) sont implémentées via Eloquent et contraintes de clés étrangères en base.

2.6 Les API REST et l'authentification

2.6.1 Principe des API REST

Une **API REST** (*Representational State Transfer*) expose des ressources (identifiées par des URL) manipulables via les verbes **HTTP** : GET (lecture), POST (création), PUT/PATCH (mise à jour), DELETE (suppression) [1]. Ce style architectural sépare clairement le *frontend* (interface utilisateur) du *backend* (logique métier et persistance), ce qui permet :

- de faire évoluer l'interface sans réécrire le serveur ;
- de réutiliser la même API pour d'éventuels clients mobiles futurs ;

- de tester indépendamment chaque couche.

Dans le projet CIVCO BTP, l'API est organisée sous le préfixe `/api/v1/` et expose des ressources métier : projets, clients, devis, factures, tâches, etc.

2.6.2 Authentification et contrôle d'accès

La sécurisation d'une application métier repose sur deux piliers :

Authentification. Laravel **Sanctum** [9] gère l'authentification par session pour les applications **SPA** : le navigateur conserve un cookie de session sécurisé, complété par une protection **CSRF** (*Cross-Site Request Forgery*) sur les requêtes mutantes. Cette approche est adaptée à une application hébergée sur le même domaine que l'API, sans la complexité d'un flux **JWT** (*JSON Web Token*) côté client.

Autorisation (RBAC). Le contrôle d'accès basé sur les rôles (**RBAC**) associe à chaque utilisateur un ensemble de permissions (`project.view`, `quote.manage`, `invoice.view`, etc.). Les routes API vérifient ces permissions avant d'exécuter l'action demandée. Ainsi, un commercial, un chef de projet et un administrateur accèdent chacun aux modules correspondant à leur profil métier.

2.7 ERP et digitalisation du secteur BTP

2.7.1 Enjeux de la gestion intégrée en BTP

Le secteur du **BTP** partage avec l'industrie des besoins de traçabilité, de planification et de coordination documentaire. Un **ERP** (*Enterprise Resource Planning*) vise à unifier ces processus : relation client, suivi de chantier, achats, facturation, ressources humaines. Dans le BTP, les spécificités suivantes renforcent l'intérêt d'un outil intégré :

- multiplicité des intervenants (MOA, BET, entreprises, sous-traitants) ;
- cycle documentaire riche (devis, bons de commande, BL, factures, avenants) ;
- suivi temporel des phases et des tâches de chantier ;
- besoin de visibilité consolidée pour la direction et les conducteurs de travaux.

2.7.2 Panorama des solutions existantes

Le marché propose des solutions generalistes et des outils spécialisés construction :

Tableau 2.6 : Panorama de solutions de gestion applicables au BTP

Solution	Type	Apport principal
SAP S/4HANA	ERP generaliste	Suite intégrée grande entreprise, modules finance et projets
Sage 100 / X3	ERP PME	Comptabilité, commercial, gestion de production
Procore [10]	SaaS BTP	Gestion de projets construction, documents, coordination
Buildertrend	SaaS BTP	Suivi chantier, relation client, devis
Solution sur mesure	Développement interne	Adaptation exacte aux processus de l'entreprise

Les ERP generalistes offrent une couverture fonctionnelle large mais imposent des coûts de licence, de paramétrage et de formation élevés. Les solutions SaaS BTP sont plus ciblées mais restent standardisées. Pour CIVCO BTP, le développement d'une **application sur mesure** permet d'aligner précisément l'outil sur les processus internes, tout en maîtrisant l'évolution fonctionnelle.

2.7.3 Digitalisation des BET au Maroc

Les bureaux d'études techniques marocains, comme CIVCO BTP, combinent expertise ingénierie et activités de suivi de chantier. La digitalisation de leurs processus internes — gestion documentaire, suivi des échéances, coordination inter-services — constitue un levier de compétitivité. Une application web interne permet de :

- réduire le temps de recherche d'information sur un projet ;
- standardiser la production des devis et factures ;
- améliorer la visibilité de l'avancement pour la direction et les conducteurs ;
- préparer une traçabilité des actions (journal d'activité, historique).

Ce contexte sectoriel justifie l'investissement dans une solution adaptée plutôt que dans un ERP generaliste surdimensionné pour la taille et les processus de l'entreprise.

2.8 Références comparatives (Benchmarks) et justification des choix

2.8.1 Comparaison des frameworks backend

Tableau 2.7 : Comparaison des frameworks backend pour une API métier

Framework	Points forts	Limites	Écosystème
Laravel	ORM Eloquent, migrations, Sanctum, rapidité de dev	Performance brute inférieure à Go/Rust	Très riche (PHP)
Symfony	Modularité, robustesse entreprise	Courbe d'apprentissage plus longue	Large (PHP)
Django	Admin intégré, ORM puissant	Moins adapté aux SPA modernes	Python
Express.js	Légereté, JavaScript full-stack	Structure laissée au développeur	Node.js

Laravel [3] ressort du Tableau 2.7 pour sa productivité, son ORM mature, son système d'authentification Sanctum adapté aux SPA et sa documentation exhaustive — critères décisifs pour un projet de stage avec délai contraint.

2.8.2 Comparaison des frameworks frontend

Tableau 2.8 : Comparaison des frameworks frontend

Framework	Popularité	Écosystème	Courbe	SPA
React	Très élevée	Très riche	Modérée	Native
Vue.js	Élevée	Riche	Faible	Native
Angular	Élevée	Complet (Google)	Élevée	Native
Svelte	Croissante	Moyen	Faible	Native

React [4] est retenu (Tableau 2.8) pour son écosystème (React Router, bibliothèques UI), sa adoption industrielle massive et sa compatibilité avec **Vite** [5] pour un cycle de développement rapide.

2.8.3 Comparaison des SGBD

Tableau 2.9 : Comparaison des systèmes de gestion de bases de données

SGBD	Open source	JSON	Performance	Déploiement
MySQL	Oui	Partiel	Élevée	Simple
PostgreSQL	Oui	Natif	Très élevée	Standard
SQLite	Oui	Oui	Faible (fichier)	Très simple
SQL Server	Non	Oui	Élevée	Entreprise

MySQL et PostgreSQL conviennent toutes deux au projet ; PostgreSQL est privilégié en production pour sa robustesse, MySQL ou SQLite pouvant servir en développement local.

2.8.4 Synthèse des choix

Tableau 2.10 : Synthèse des choix technologiques et critères décisifs

Composant	Choix retenu	Critère décisif
Backend	Laravel 13 / PHP 8.3	Productivité, ORM, écosystème mature
Frontend	React 19 + Vite 8	SPA réactive, écosystème riche
Style UI	Tailwind CSS 4 [6]	Développement rapide, cohérence visuelle
API	REST / JSON	Simplicité, séparation frontend/backend
Authentification	Laravel Sanctum	Sessions SPA sécurisées, CSRF
Base de données	MySQL / PostgreSQL	Modèle relationnel, intégrité
ORM	Eloquent	Mapping objet-relationnel, migrations
Contrôle d'accès	RBAC (rôles/permissions)	Profils métier CIVCO BTP

Le Tableau 2.10 récapitule l'ensemble des choix retenus pour l'application de gestion de projets de CIVCO BTP.

2.9 Méthodologies de développement logiciel

2.9.1 Approches traditionnelles et agiles

Le développement d'une application métier peut suivre différentes méthodologies. L'approche **cascade** (*Waterfall*) enchaîne les phases de manière séquentielle : analyse, conception, développement, tests, déploiement. Elle convient aux projets dont les besoins sont stables et parfaitement définis en amont, mais peut s'avérer rigide lorsque les exigences évoluent en cours de réalisation.

Les méthodes **agiles** [11], à l'inverse, privilégient des itérations courtes (sprints de deux à quatre semaines), une collaboration continue avec le client et l'adaptation au changement.

Le **Manifeste Agile** (2001) formalise quatre valeurs : individus et interactions plutôt que processus et outils, logiciel fonctionnel plutôt que documentation exhaustive, collaboration client plutôt que négociation contractuelle, adaptation au changement plutôt que suivi d'un plan rigide.

2.9.2 Application au projet CIVCO BTP

Dans le contexte de ce stage, une approche **itérative et incrémentale** a été retenue, s'inspirant des principes agiles. Le suivi des tâches a été matérialisé via **Trello**, organisé selon un tableau **Kanban** (colonnes *À faire*, *En cours*, *Terminé*), sans appliquer un cadre Scrum formel (sprints, daily meetings). Les raisons de ce choix sont les suivantes :

- **Périmètre évolutif** : les besoins métier ont été affinés au fil des démonstrations, notamment pour les modules d'impression et de portail client.
- **Durée limitée** : quatre mois de stage imposent des livrables réguliers plutôt qu'une phase d'analyse prolongée.
- **Proximité métier** : l'encadrant professionnel était disponible pour valider chaque incrément et orienter les priorités.
- **Risque maîtrisé** : chaque module livré est testable indépendamment, réduisant le risque d'une intégration finale décevante.

2.9.3 Qualité logicielle et bonnes pratiques

Indépendamment de la méthodologie, plusieurs principes de qualité logicielle ont guidé le développement :

- **Principe DRY** (*Don't Repeat Yourself*) : factorisation de la logique commune dans des services et des composants réutilisables.
- **Principe SOLID** : responsabilité unique des classes (un service par domaine métier), injection de dépendances via le conteneur Laravel.
- **Conventions de code** : Laravel Pint pour le PHP, ESLint pour le JavaScript, respect des standards PSR-12 et des conventions React.
- **Tests automatisés** : couverture des flux critiques (authentification, CRUD, conversion devis) par PHPUnit pour détecter les régressions.
- **Documentation** : commentaires sur les règles métier complexes, présent rapport et annexes techniques.

Ces pratiques, combinées à l'architecture modulaire décrite aux chapitres suivants, visent à produire un code maintenable et évolutif, au-delà de la seule durée du stage.

Conclusion

Ce chapitre a posé les fondements théoriques et technologiques sur lesquels repose l'application développée pour CIVCO BTP. Nous avons d'abord rappelé l'évolution des applications web vers des architectures *full-stack* en trois tiers, dans lesquelles la couche présentation (React), la couche métier (Laravel) et la couche données (SGBD relationnel) sont clairement séparées. Ce découplage facilite la maintenance, les tests et l'évolution indépendante de chaque composant.

L'étude de **Laravel** a montré comment un framework PHP mature structure le *backend* autour du patron **MVC**, d'Eloquent, de migrations et de Sanctum. Les middleware, les Form Requests et les API Resources garantissent à la fois la sécurité, la validation des données et un format de réponse JSON cohérent pour l'ensemble des modules métier.

Côté interface, **React** permet de construire une **SPA** modulaire et réactive, organisée par fonctionnalités et appuyée sur un écosystème riche (Vite, Axios, Tailwind CSS, Recharts, Leaflet). L'échange permanent entre le navigateur et l'API REST via JSON constitue le lien central de l'architecture retenue.

Le modèle relationnel et l'**ORM** assurent l'intégrité et la cohérence des données — clients, projets, phases, tâches, devis, factures et contrats — tandis que le **RBAC** adapte l'accès aux différents profils métier de l'entreprise. Enfin, le panorama des solutions ERP et BTP a situé le projet dans son contexte sectoriel et confirmé la pertinence d'une application sur mesure pour CIVCO BTP.

Les benchmarks comparatifs ont permis de justifier chaque choix : Laravel pour la productivité backend, React pour l'expérience utilisateur, PostgreSQL/MySQL pour la persistance, Sanctum pour l'authentification SPA. L'ensemble de ces décisions est guidé par trois impératifs : la **productivité** de développement dans le cadre du stage, la **maintenabilité** du code sur le long terme, et l'**adéquation métier** aux processus internes de l'entreprise.

Le chapitre suivant traduit ces choix en une architecture système concrète : analyse des besoins, identification des acteurs, cas d'utilisation et modélisation **UML** de l'application de gestion et de planification des projets de construction.

Chapitre 3 : Conception du Système

3.1 Introduction

Ce chapitre présente la phase de conception de l'application développée pour CIVCO BTP. Il traduit les besoins métier identifiés au Chapitre 1 en spécifications techniques formelles et en modèles architecturaux. Nous commençons par identifier les acteurs et leurs cas d'utilisation, puis nous définissons les exigences fonctionnelles et non-fonctionnelles du système. Nous décrivons ensuite l'architecture retenue, la conception des modules métier principaux, et nous terminons par la modélisation **UML** : diagrammes de cas d'utilisation, de séquence, de classes et d'activité.

3.2 Analyse des besoins

3.2.1 Identification des acteurs

Six catégories d'acteurs interagissent avec l'application de gestion de CIVCO BTP :

Administrateur Responsable de la configuration globale de l'application : gestion des utilisateurs, des rôles et permissions, des paramètres documentaires, du thème visuel et de la consultation de l'historique des actions.

Chef de projet Pilote un ou plusieurs chantiers : suivi des phases et des tâches, affectation des équipes, consultation des documents liés au projet, gestion des contrats et des avenants.

Commercial Gère la relation client et les documents commerciaux : création et suivi des devis, conversion en factures, production des bons de livraison.

Comptable Assure le suivi financier : consultation et validation des factures, suivi des paiements, état de facturation par projet.

Utilisateur interne Collaborateur terrain ou technique assigné à des tâches spécifiques : consultation de ses missions, mise à jour de l'avancement, accès limité aux projets auxquels il est affecté.

Client externe Maître d'ouvrage ou client final accédant au portail dédié pour consulter l'état de ses projets, accepter les devis, signer les contrats et échanger avec l'équipe CIVCO BTP.

3.2.2 Cas d'utilisation

Les cas d'utilisation principaux, regroupés par domaine fonctionnel, sont les suivants :

Authentification et tableau de bord :

- UC-01 : S'authentifier à l'application (connexion sécurisée par session).
- UC-02 : Consulter le tableau de bord synthétique (projets en cours, tâches, indicateurs clés).

Gestion des projets :

- UC-03 : Créer et modifier un projet de construction (référence, client, budget, dates, localisation).
- UC-04 : Définir les phases d'un projet et suivre l'avancement global.
- UC-05 : Affecter des membres d'équipe à un projet.
- UC-06 : Consulter la carte géographique des chantiers.

Gestion des clients :

- UC-07 : Créer, modifier et archiver une fiche client.
- UC-08 : Gérer les contacts associés à un client.
- UC-09 : Provisionner un accès portail pour un client externe.

Documents commerciaux :

- UC-10 : Créer un devis avec lignes de détail et le lier à un projet.
- UC-11 : Convertir un devis accepté en facture.
- UC-12 : Émettre un bon de livraison (**BL**) lié à un projet.
- UC-13 : Imprimer ou exporter un document commercial (devis, facture, BL).

Contrats et tâches :

- UC-14 : Créer et suivre un contrat associé à un projet.
- UC-15 : Enregistrer un avenant contractuel (modification budget/durée).

- UC-16 : Créer, assigner et suivre des tâches opérationnelles.

Administration et portail client :

- UC-17 : Configurer les rôles, permissions et paramètres de l'application.
- UC-18 : Consulter le journal d'activité et l'historique des actions.
- UC-19 : Échanger des messages avec un client via la messagerie intégrée.
- UC-20 : (Client externe) Consulter ses projets, accepter un devis, signer un contrat via le portail.

3.2.3 Exigences fonctionnelles

- **EF-01 Authentification** : Connexion par email et mot de passe via Laravel Sanctum (session sécurisée par cookie, protection **CSRF**).
- **EF-02 Gestion des rôles** : Attribution de rôles métier (administrateur, chef de projet, commercial, comptable, utilisateur) avec permissions granulaires (**RBAC**).
- **EF-03 Gestion des projets** : **CRUD** complet sur les projets, avec phases, équipe, budget, avancement, géolocalisation du chantier.
- **EF-04 Gestion des clients** : Fiches clients, contacts, archivage, provisionnement d'accès portail.
- **EF-05 Devis** : Création de devis multi-lignes, calcul automatique des totaux HT/TVA/TTC, suivi des statuts (brouillon, envoyé, accepté, refusé).
- **EF-06 Factures** : Conversion devis → facture, suivi de l'état de paiement, numérotation automatique.
- **EF-07 Bons de livraison** : Production de BL liés aux projets, avec lignes de détail et suivi des quantités livrées.
- **EF-08 Contrats** : Gestion des contrats et avenants, compilation à partir de modèles, double signature (entreprise + client).
- **EF-09 Tâches** : Planification par phase, assignation, statuts (à faire, en cours, terminé), filtrage par utilisateur.
- **EF-10 Tableau de bord** : Widgets configurables (projets actifs, tâches prioritaires, graphiques d'avancement).
- **EF-11 Portail client** : Interface dédiée pour consultation des projets, acceptation de devis et signature de contrats.

- **EF-12 Messagerie** : Échanges projet-scopés entre équipe interne et client externe.
- **EF-13 Historique** : Journalisation des actions sensibles (création, modification, archivage) consultable par l'administrateur.

3.2.4 Exigences non-fonctionnelles

Performance :

- **ENF-01** : Temps de réponse des API inférieur à 500 ms pour les opérations de lecture courantes (listes, fiches détail).
- **ENF-02** : Chargement initial de l'interface (SPA) inférieur à 3 s sur une connexion standard.

Sécurité :

- **ENF-03** : Mots de passe hachés (bcrypt); sessions Sanctum avec expiration configurable.
- **ENF-04** : Contrôle d'accès par permission sur chaque route API; vérification côté serveur avant toute action métier.
- **ENF-05** : Protection **CSRF** sur les requêtes mutantes; communication **HTTPS** en production.

Ergonomie et accessibilité :

- **ENF-06** : Interface responsive adaptée aux postes de bureau et tablettes de chantier.
- **ENF-07** : Support bilingue français / anglais via fichiers de traduction centralisés.

Maintenabilité :

- **ENF-08** : Architecture modulaire : contrôleurs fins, services métier, modèles Eloquent, composants React par feature.
- **ENF-09** : Schéma de base versionné par migrations Laravel, reproductible entre environnements.

3.2.5 Scénarios d'utilisation détaillés

Pour illustrer concrètement les exigences fonctionnelles, trois scénarios types ont été formalisés lors de la phase de conception :

Scénario 1 — Lancement d’un nouveau chantier : un chef de projet crée un projet en sélectionnant le client, renseigne le budget et les dates, définit les phases (études, terrassement, gros œuvre, finitions), affecte l’équipe (ingénieur, conducteur de travaux, technicien) et positionne le chantier sur la carte. Le commercial rattache un devis préalablement validé.

Scénario 2 — Cycle commercial complet : le commercial crée un devis multi-lignes pour un projet existant, l’envoie au client via le portail, le client l’accepte en ligne, le comptable convertit le devis en facture, enregistre un paiement partiel puis le solde. Chaque étape est tracée dans le journal d’activité.

Scénario 3 — Suivi opérationnel quotidien : un conducteur de travaux consulte ses tâches du jour sur tablette, met à jour le statut d’une tâche terminée, le chef de projet ajuste l’avancement global du projet et le tableau de bord reflète immédiatement la progression.

3.3 Architecture du système

3.3.1 Architecture globale en trois couches

L’application adopte une architecture trois couches, avec une séparation nette entre présentation, logique métier et persistance :

Couche Présentation : React 19 + Vite + Tailwind CSS

Interface web organisée en **SPA** : tableau de bord, modules projets, clients, devis, factures, tâches, configuration et portail client. Navigation par React Router, état global via Context API (authentification, thème, langue).

Couche Logique métier : Laravel 13 + Sanctum

API **REST** sous `/api/v1/` : contrôleurs par domaine, services métier, validation (Form Requests), transformation JSON (API Resources), middleware de permissions et journalisation d’activité.

Couche Persistance : MySQL / PostgreSQL + Eloquent ORM

Stockage relationnel de toutes les entités métier : utilisateurs, rôles, clients, projets, documents commerciaux, contrats, tâches. Migrations versionnées et seeders de démonstration.

Les échanges entre ces couches s’effectuent en **JSON** via **HTTPS** côté présentation, et en **SQL** via l’**ORM** Eloquent côté persistance.

3.3.2 Organisation du backend Laravel

Le backend est structuré selon le patron **MVC** étendu par une couche de services :

- **Contrôleurs** (app/Http/Controllers/Api/V1/) : points d'entrée HTTP, délégation aux services, retour de Ressources JSON.
- **Form Requests** (app/Http/Requests/) : validation des données entrantes par domaine (StoreProjectRequest, StoreQuoteRequest, etc.).
- **Services** (app/Services/) : logique métier complexe (QuoteService, InvoiceService, DocumentService, DashboardService, etc.).
- **Modèles** (app/Models/) : entités Eloquent avec relations, casts et scopes.
- **Resources** (app/Http/Resources/) : sérialisation JSON normalisée pour le frontend.
- **Middleware** : authentification Sanctum, vérification des permissions, résolution du contexte entreprise.

3.3.3 Organisation du frontend React

Le frontend suit une organisation par **features** (domaines métier) :

- features/projects/ : liste, fiche détail, onglets phases/tâches/équipe.
- features/clients/ : gestion clients, portail, archivage.
- features/quotes/, invoices/, delivery-forms/ : documents commerciaux.
- features/tasks/ : planification et suivi des tâches.
- features/dashboard/ : tableau de bord et widgets.
- features/configuration/ : paramètres, rôles, modèles de documents.
- api/ : modules client HTTP (Axios) par ressource API.

3.3.4 Protocoles de communication

L'application repose principalement sur le protocole **REST** (HTTP/JSON) :

- **Authentification** : flux Sanctum SPA (cookie de session + jeton CSRF).
- **Opérations CRUD** : verbes HTTP standard sur les ressources métier (GET /api/v1/projects, POST /api/v1/quotes, etc.).

- **Upload de fichiers** : requêtes multipart/form-data pour les documents joints, médias de projet et signatures.
- **Impression** : génération de PDF côté client (composants React dédiés) avec suivi des impressions officielles.

3.3.5 Flux d'authentification

Le flux d'authentification Sanctum pour une SPA suit les étapes suivantes :

1. Le frontend appelle GET /sanctum/csrf-cookie pour obtenir un jeton **CSRF**.
2. L'utilisateur saisit ses identifiants; le frontend envoie POST /api/v1/login avec le jeton CSRF.
3. Laravel valide les identifiants, crée une session et retourne le profil utilisateur (rôles, permissions).
4. Les requêtes suivantes incluent automatiquement le cookie de session; chaque route protégée vérifie l'authentification et les permissions.
5. En cas de session expirée (401), le frontend redirige vers la page de connexion.

3.3.6 Matrice des permissions par profil

Le Tableau 3.1 présente une synthèse des droits d'accès par profil métier. Les permissions exactes sont configurables par l'administrateur; le tableau reflète la configuration type déployée pour CIVCO BTP.

Tableau 3.1 : Matrice des permissions par profil utilisateur (synthèse)

Module / Action	Admin	Chef proj.	Com.	Compt.	Utilis.
Tableau de bord	✓	✓	✓	✓	✓
Projets (lecture)	✓	✓	✓	✓	✓*
Projets (écriture)	✓	✓	—	—	—
Clients	✓	✓	✓	—	—
Devis	✓	✓	✓	—	—
Factures	✓	✓	✓	✓	—
Tâches (toutes)	✓	✓	—	—	—
Tâches (propres)	✓	✓	✓	✓	✓
Configuration	✓	—	—	—	—
Historique	✓	—	—	—	—

* Accès limité aux projets assignés.

3.4 Conception des modules métier

Cette section détaille la conception fonctionnelle des principaux modules de l'application, en précisant les entités manipulées, les règles de gestion et les interactions entre modules.

3.4.1 Module Projets

Le module Projets constitue le cœur de l'application. Un projet représente un chantier ou une affaire de construction, lié à un client, découpé en phases et suivi par une équipe interne.

Entités principales : Project, ProjectPhase, Expense, ProjectMedia, ProgressSnapshot.

Règles de gestion :

- Chaque projet possède une référence unique et un statut (planifié, en cours, suspendu, terminé, archivé).
- L'avancement (progress_percent) peut être mis à jour manuellement ou calculé à partir des tâches terminées.
- La géolocalisation (latitude/longitude) permet l'affichage sur la carte des chantiers (Leaflet).
- Seuls les utilisateurs disposant de la permission `project.manage` peuvent créer ou modifier un projet.

3.4.2 Module Clients

Le module Clients centralise les informations des maîtres d'ouvrage et clients commerciaux de CIVCO BTP.

Entités principales : Client, ClientContact, Badge.

Règles de gestion :

- Un client peut être archivé sans suppression des projets associés (soft archive).
- La création d'un accès portail génère un compte utilisateur lié au client (client_id sur User).
- Les contacts multiples sont gérés via la relation ClientContact.

3.4.3 Module Documents commerciaux

Ce module regroupe les devis, factures et bons de livraison, qui partagent une structure commune de lignes de détail.

Entités principales : Quote, QuoteLine, Invoice, InvoiceLine, DeliveryForm, DeliveryFormLine, Payment.

Flux documentaire :

1. Le commercial crée un devis (statut *brouillon*) lié à un projet.
2. Le devis est validé et envoyé au client (statut *envoyé*).
3. Le client accepte via le portail ou l'équipe interne confirme l'acceptation.
4. Le devis est converti en facture (copie des lignes, nouvelle référence).
5. Les paiements partiels ou totaux sont enregistrés sur la facture.
6. Les bons de livraison documentent les livraisons matérielles sur chantier.

3.4.4 Module Contrats

Le module Contrats permet de formaliser l'engagement contractuel lié à un projet, avec support des avenants.

Entités principales : Contract, ContractAmendment, ContractTemplate, DocumentTemplate.

Règles de gestion :

- Un contrat est généré à partir d'un modèle (ContractTemplate) et des données du projet (compilation de champs dynamiques).
- Les avenants modifient le budget ou la durée sans recréer le contrat initial.
- La signature client est capturée via un pad de signature sur le portail.

3.4.5 Module Tâches

Le module Tâches organise le travail opérationnel au sein de chaque projet.

Entités principales : Task (liée à Project et optionnellement à ProjectPhase).

Règles de gestion :

- Une tâche est assignée à un utilisateur et possède un statut, une priorité et une date d'échéance.

- Les permissions `task.view_own` et `task.view_all` contrôlent la visibilité selon le profil.
- Le tableau de bord agrège les tâches en retard et à venir.

3.4.6 Module Tableau de bord et carte

Le tableau de bord offre une vue consolidée de l'activité de CIVCO BTP :

- Nombre de projets actifs, en retard, terminés.
- Graphiques d'avancement (Recharts).
- Widget des tâches prioritaires de l'utilisateur connecté.
- Carte Leaflet des projets géolocalisés.

Les widgets sont conditionnés par les permissions de l'utilisateur : un commercial voit les indicateurs commerciaux, un chef de projet les indicateurs de chantier.

3.4.7 Module Portail client et messagerie

Le portail client constitue l'interface dédiée aux clients externes de CIVCO BTP. Accessible via une route séparée (`/portal`), il offre une expérience simplifiée centrée sur la consultation et la validation.

Fonctionnalités portail :

- Tableau de bord client : projets en cours, documents en attente de signature.
- Consultation des devis et acceptation en ligne.
- Signature électronique des contrats (pad de signature canvas).
- Fil de discussion avec l'équipe projet (si autorisé).
- Calendrier des échéances et jalons du projet.

Messagerie interne : Les échanges entre l'équipe CIVCO BTP et le client sont scopés par projet. Seuls les membres d'équipe disposant de l'autorisation `can_chat_with_client` peuvent initier ou répondre à une conversation. Les messages sont persistés en base (`PortalMessage`) et consultables depuis l'interface d'administration.

3.4.8 Module Configuration et historique

Le module Configuration regroupe les paramètres transverses de l'application :

- **Rôles et permissions** : création de profils métier, attribution de permissions granulaires (`project.view`, `quote.manage`, etc.).
- **Modèles de documents** : templates de contrats et documents commerciaux avec champs dynamiques (placeholders).
- **Contrôles d'impression** : quotas d'impressions officielles, filigrane pour les copies non officielles.
- **Thème visuel** : couleurs de l'interface, logo de l'entreprise.
- **Référentiels** : badges clients, lots, secteurs d'activité.

Le module Historique (ActivityLog) enregistre les actions sensibles : création/modification de projets, validation de devis, archivage de clients. Chaque entrée contient l'utilisateur, l'action, l'entité concernée et l'horodatage, offrant une traçabilité utile à l'administration et à l'audit interne.

3.5 Modélisation de la base de données

3.5.1 Principes de conception

La base de données relationnelle constitue le socle de persistance de l'application. Sa conception repose sur les principes suivants :

- **Normalisation** : les entités sont décomposées pour éviter la redondance (clients séparés des projets, lignes de devis dans une table dédiée).
- **Intégrité référentielle** : les clés étrangères garantissent la cohérence (un devis appartient toujours à un projet et un client existants).
- **Versionnement** : le schéma évolue via des migrations Laravel, reproductibles entre environnements de développement, test et production.
- **Traçabilité** : les horodatages (`created_at`, `updated_at`) et le journal d'activité complètent l'historique métier.

3.5.2 Tables principales

Le Tableau 3.2 présente les tables centrales du modèle de données et leur rôle dans l'application.

Tableau 3.2 : Tables principales de la base de données

Table	Entité	Rôle
users	Utilisateur	Comptes internes et clients portail
roles, permissions	Sécurité	Profils métier et droits granulaires
clients	Client	Maîtres d'ouvrage et clients commerciaux
projects	Projet	Chantiers, budget, avancement, coordonnées
project_phases	Phase	Découpage temporel d'un projet
tasks	Tâche	Missions opérationnelles assignées
quotes, quote_lines	Devis	Documents commerciaux et lignes
invoices, invoice_lines	Facture	Facturation et détail des prestations
contracts	Contrat	Engagements contractuels liés aux projets
activity_logs	Journal	Traçabilité des actions sensibles

Au total, le schéma compte plus de soixante-dix tables lorsque l'on inclut les tables de liaison (équipe projet, badges clients, paiements, impressions, etc.), les référentiels (lots, secteurs) et les tables techniques (sessions, notifications).

3.5.3 Relations et contraintes d'intégrité

Les relations cardinales les plus importantes structurent le modèle métier :

- Un **client** possède zéro à plusieurs **projets** ; un projet est rattaché à un seul client.
- Un **projet** regroupe plusieurs **phases**, **tâches**, **devis**, **factures** et **contrats**.
- Un **devis accepté** peut être converti en une seule **facture** (relation 1:0..1), les lignes étant recopiées automatiquement.
- Un **utilisateur** peut appartenir à l'équipe de plusieurs projets (relation N :N via project_user).
- Un **rôle** agrège plusieurs **permissions** ; chaque utilisateur interne est associé à un rôle métier.

Les contraintes ON DELETE RESTRICT sur les clés étrangères critiques (client, projet) empêchent la suppression accidentelle d'enregistrements encore référencés. L'archivage des clients et la clôture des projets sont privilégiés à la suppression physique.

3.6 Conception de l'interface utilisateur

3.6.1 Principes ergonomiques

L'interface a été conçue pour un usage professionnel quotidien, en tenant compte des contraintes des utilisateurs de CIVCO BTP :

- **Lisibilité** : thème sombre par défaut, contrastes élevés, typographie claire pour une utilisation prolongée devant écran.
- **Efficacité** : accès rapide aux modules via une barre latérale fixe, recherche globale et raccourcis depuis le tableau de bord.
- **Cohérence** : chaque module suit le même patron (liste → fiche détail → actions contextuelles).
- **Responsive** : adaptation aux tablettes utilisées sur chantier pour la consultation des tâches et des documents.
- **Feedback utilisateur** : messages de confirmation, indicateurs de chargement et alertes en cas d'erreur de validation.

3.6.2 Charte graphique et navigation

L'identité visuelle de CIVCO BTP est intégrée via un système de thème dynamique : couleur principale (ensamBlue / bleu entreprise), logo en en-tête et variables CSS injectées depuis la configuration. La navigation repose sur une **sidebar** persistante dont les entrées sont filtrées selon les permissions de l'utilisateur connecté.

La structure de navigation distingue trois zones :

1. **Zone opérationnelle** : tableau de bord, projets, clients, tâches, carte des chantiers.
2. **Zone documentaire** : devis, factures, bons de livraison.
3. **Zone administration** : configuration, historique, gestion d'équipe (réservée aux profils autorisés).

Le portail client adopte une interface simplifiée, sans la sidebar interne, centrée sur la consultation et la validation des documents.

3.6.3 Parcours utilisateurs types

Trois parcours types ont guidé la conception des écrans :

Parcours commercial : connexion → création d'un client → création d'un projet → production d'un devis → envoi au client → conversion en facture après acceptation.

Parcours chef de projet : connexion → consultation du tableau de bord → ouverture d'un chantier → mise à jour des phases et des tâches → affectation des collaborateurs → suivi de l'avancement sur la carte.

Parcours client externe : connexion portail → consultation des projets en cours → acceptation d'un devis → signature électronique du contrat → échange via la messagerie intégrée.

3.7 Conception de la sécurité et de la traçabilité

3.7.1 Modèle de menaces et contre-mesures

La conception de la sécurité s'appuie sur une analyse des risques les plus probables dans le contexte d'une application métier interne :

Tableau 3.3 : Menaces identifiées et contre-mesures associées

Menace	Impact	Contre-mesure
Accès non autorisé	Consultation ou modification de données sensibles	Sanctum, RBAC, middleware permissions
Falsification de requête (CSRF)	Action non voulue au nom d'un utilisateur	Jeton CSRF sur requêtes mutantes
Élévation de privilèges	Contournement des restrictions métier	Vérification côté serveur sur chaque route
Fuite de données client	Atteinte à la confidentialité	Filtrage par projet et par rôle
Perte de traçabilité	Impossibilité d'auditer les actions	Journal d'activité (ActivityLog)

3.7.2 Journalisation des actions

Le module ActivityLogService enregistre les opérations sensibles : création ou modification d'un projet, validation d'un devis, archivage d'un client, changement de rôle. Chaque entrée comprend l'identifiant de l'utilisateur, le type d'action, l'entité concernée, l'horodatage et éventuellement les valeurs modifiées. L'administrateur consulte ce journal depuis le module Historique.

3.7.3 Gestion des impressions officielles

Les documents commerciaux (devis, factures, bons de livraison) peuvent être imprimés en version **officielle** (avec en-tête CIVCO BTP) ou en **copie non officielle**. Un quota configurable limite le nombre d'impressions officielles par type de document. Au-delà du quota, le système applique automatiquement un filigrane «COPIE - NON OFFICIEL» et enregistre chaque impression dans la table de suivi (print_trackings).

3.8 Modélisation UML

3.8.1 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation (Figure 3.1) représente les interactions entre les six acteurs identifiés et le système. L'administrateur dispose des droits les plus étendus (configuration, historique, gestion des utilisateurs). Le client externe accède uniquement au portail pour consulter ses projets et valider les documents qui le concernent.

Les relations entre acteurs et cas d'utilisation suivent la logique métier de CIVCO BTP : le commercial intervient sur les clients et les devis, le comptable sur les factures, le chef de projet sur les chantiers et les tâches. Les cas d'utilisation UC-01 à UC-20 détaillés en section 1.2 constituent la base de ce diagramme. Les cas d'utilisation *include* (par exemple, «S'authentifier» précède toute action protégée) et *extend* (par exemple, «Imprimer un document» en complément de «Consulter un devis») structurent les dépendances entre fonctionnalités.

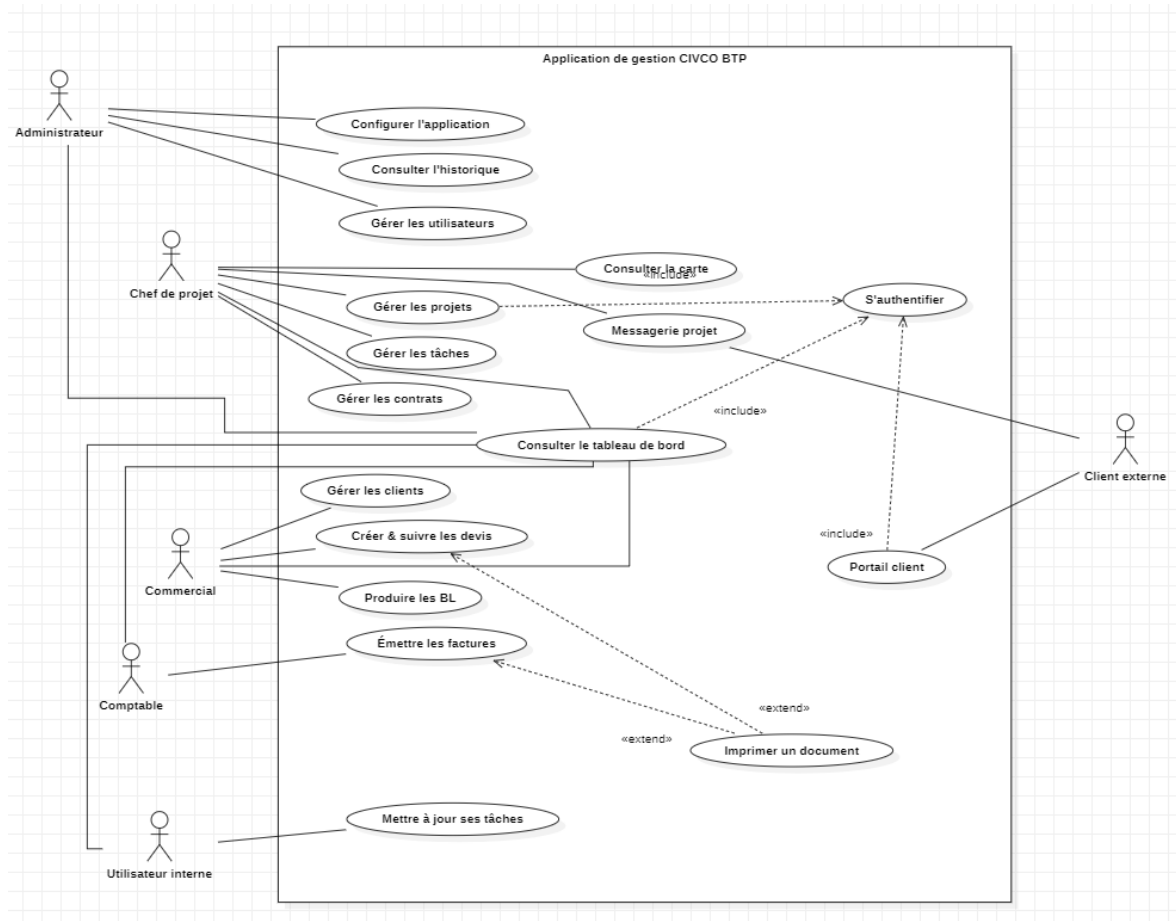


Figure 3.1 : Diagramme de cas d'utilisation de l'application CIVCO BTP

3.8.2 Diagramme de séquence

Le diagramme de séquence (Figure 3.2) illustre le scénario de création d'un devis par un commercial. Les interactions mettent en évidence la séparation entre l'interface React, l'API Laravel (validation, service métier) et la persistance en base de données. Les étapes principales sont les suivantes :

1. Le commercial s'authentifie via la SPA (cookie Sanctum + jeton CSRF).
2. Il sélectionne un projet et saisit les lignes du devis.
3. Le frontend envoie POST /api/v1/quotes avec les données JSON.
4. Laravel valide la requête (Form Request), persiste le devis et ses lignes.
5. L'API retourne une `QuoteResource` ; l'interface affiche la confirmation.

Ce flux est représentatif du patron adopté sur l'ensemble des modules : le frontend ne contient aucune logique métier critique ; la validation, les calculs et la persistance sont toujours assurés côté serveur.

D'autres scénarios de séquence ont été modélisés selon le même principe : **conversion devis** → **facture** (contrôle du statut, transaction base de données, copie des lignes), **signature de contrat** (génération du PDF, capture de la signature canvas, mise à jour du statut) et **assignation de tâche** (notification de l'utilisateur concerné).

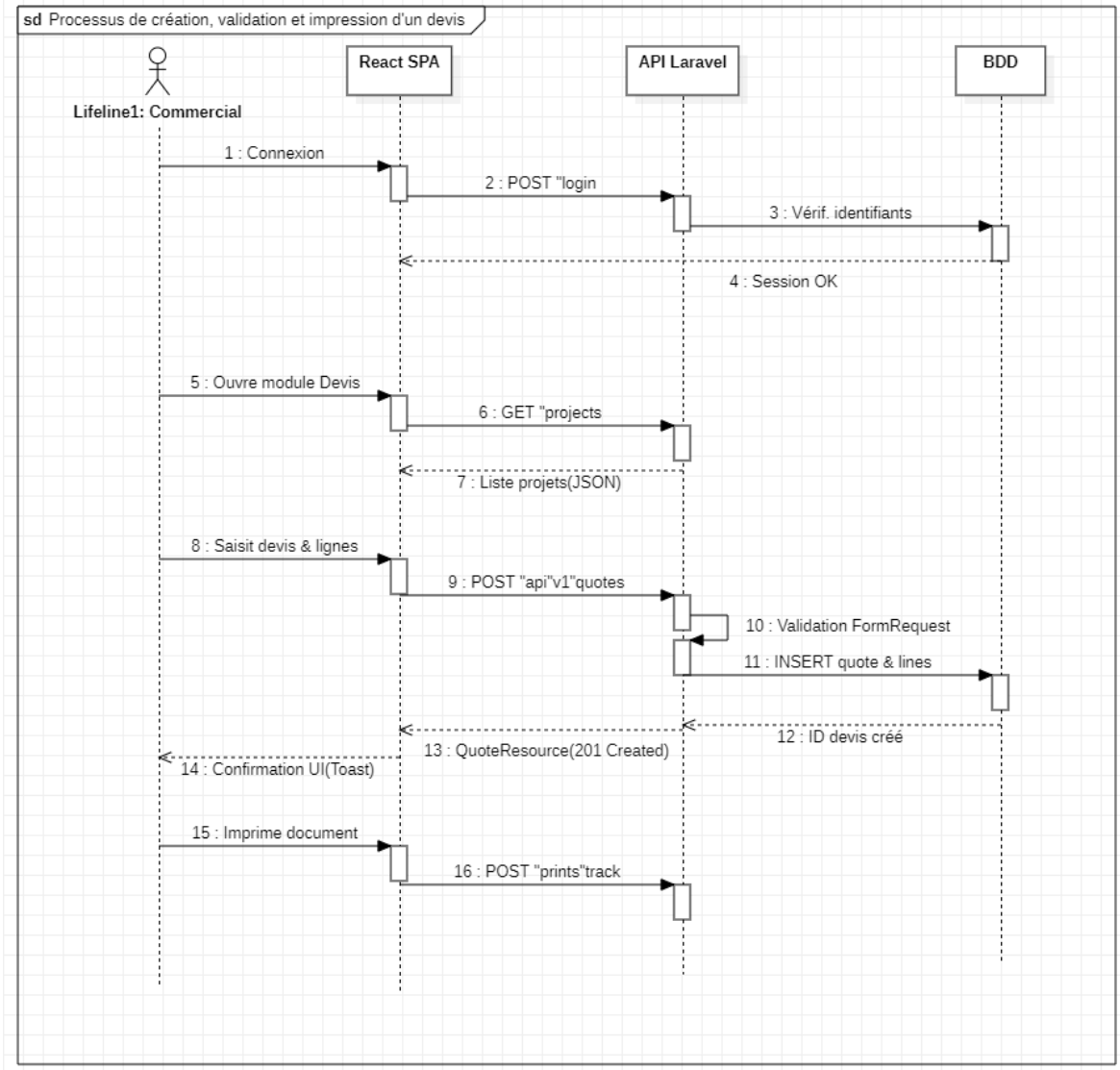


Figure 3.2 : Diagramme de séquence : création et enregistrement d'un devis

3.8.3 Diagramme de classes

Le diagramme de classes (Figure 3.3) représente le modèle de données persistant simplifié. Le Tableau 3.4 complète la description des entités principales et de leurs relations.

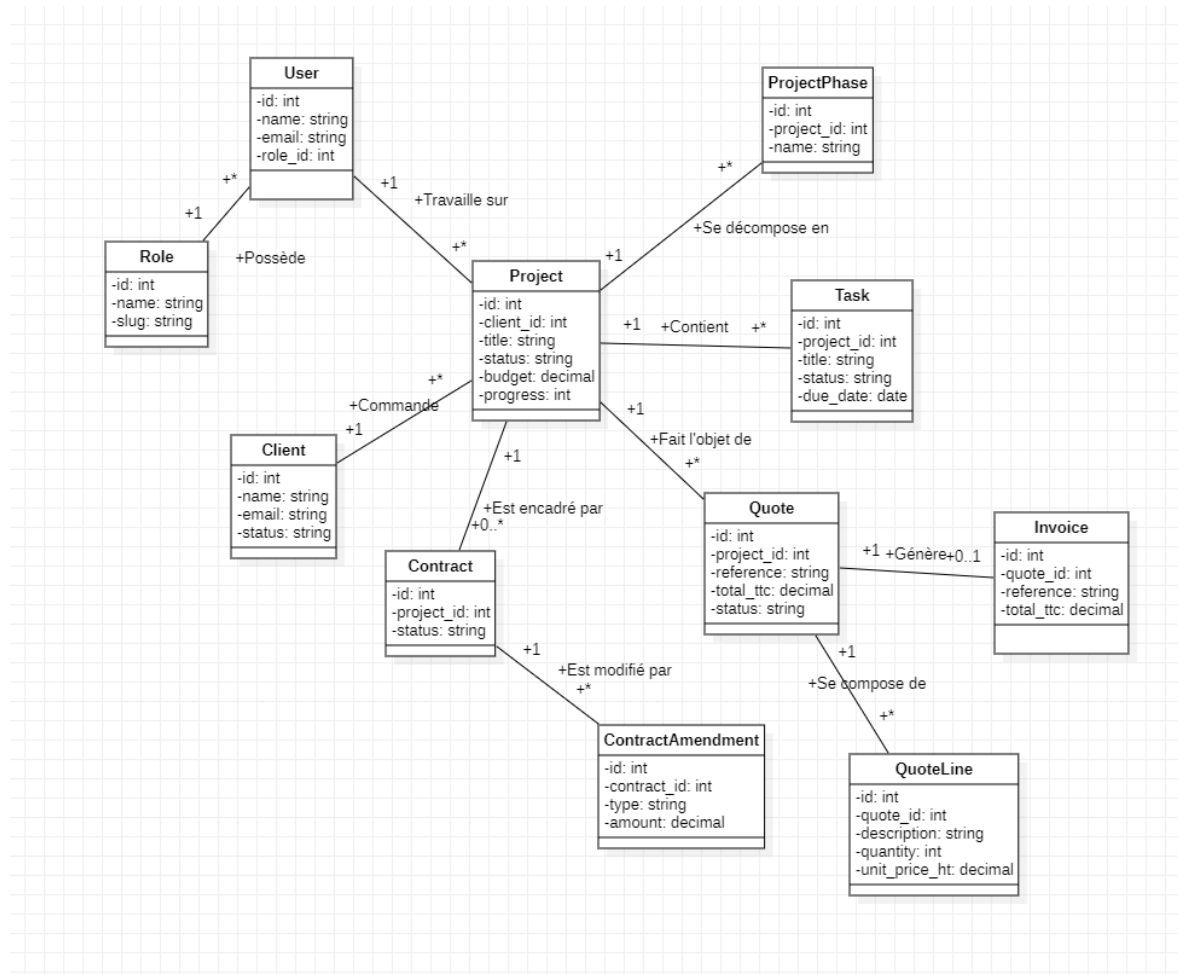


Figure 3.3 : Diagramme de classes simplifié du modèle de données métier

Tableau 3.4 : Description des classes et de leurs relations

Classe	Attributs principaux	Relations
User	id, name, email, role_id	N User $\rightarrow$ 1 Role; N $\leftrightarrow$ N Project (équipe)
Role	id, name, slug	1 Role $\rightarrow$ N Permission
Client	id, name, email, status	1 Client $\rightarrow$ N Project; 1 $\rightarrow$ N ClientContact
Project	id, client_id, title, status, budget, progress	1 $\rightarrow$ N Phase, Task, Quote, Contract
Quote	id, project_id, reference, total_ttc, status	1 $\rightarrow$ N QuoteLine; 1 $\rightarrow$ 0..1 Invoice
Invoice	id, quote_id, reference, total_ttc	1 $\rightarrow$ N InvoiceLine, Payment
Task	id, project_id, title, status, due_date	N $\rightarrow$ 1 Project; N $\rightarrow$ 1 User (assigné)
Contract	id, project_id, status	1 $\rightarrow$ N ContractAmendment

Le modèle met en évidence la **centralité du projet** : la plupart des entités métier gravitent autour de Project, ce qui reflète l'organisation interne de CIVCO BTP où le chantier constitue l'unité de travail de référence. Les énumérations (ProjectStatus, QuoteStatus, InvoiceStatus) garantissent la cohérence des états dans l'application.

3.8.4 Diagramme d'activité

Le diagramme d'activité (Figure 3.4) modélise le cycle de vie d'un projet de construction, depuis sa création jusqu'à sa clôture. Le flux principal comprend : création du projet et affectation du client, constitution de l'équipe et définition des phases, planification des tâches, établissement du devis, signature du contrat, exécution des travaux avec suivi d'avancement, facturation et clôture. Les points de décision portent sur l'acceptation du devis et l'achèvement du projet ; en cas de refus, le devis peut être révisé avant reprise du flux.

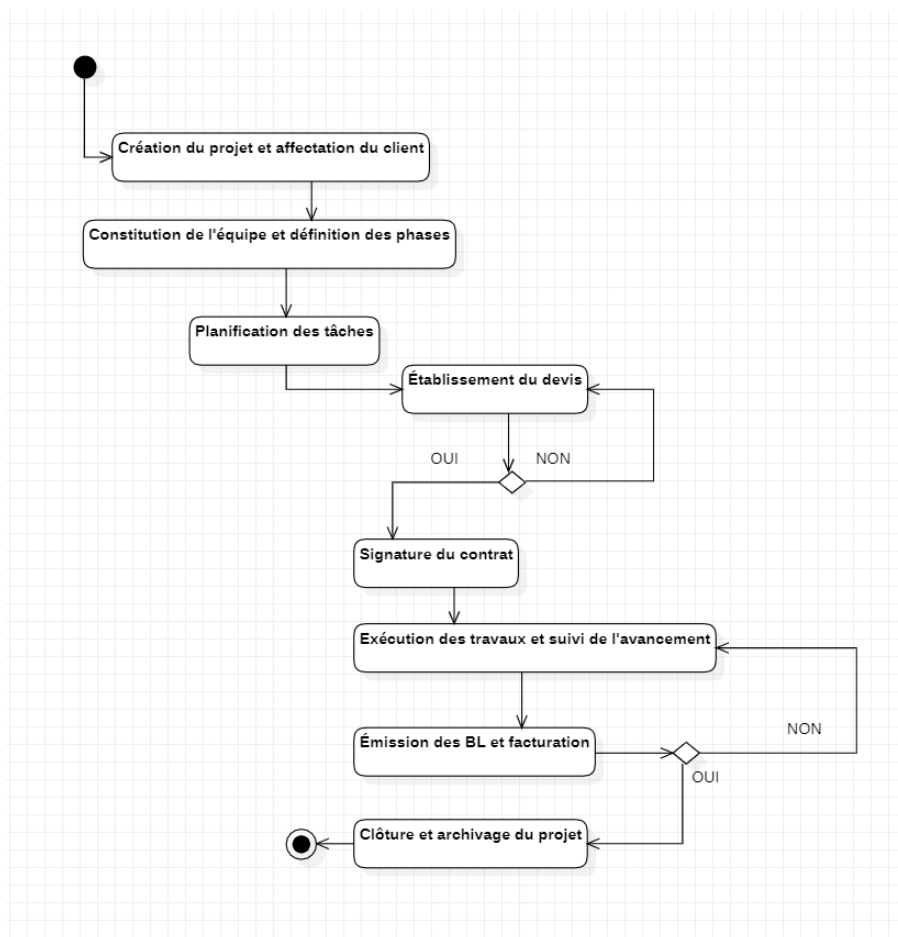


Figure 3.4 : Diagramme d'activité : cycle de vie d'un projet de construction

Conclusion

Ce chapitre a formalisé la conception de l'application de gestion de projets de CIVCO BTP à travers trois niveaux d'abstraction complémentaires.

Au niveau **fonctionnel**, l'analyse des besoins a identifié six acteurs métier, vingt cas d'utilisation et treize exigences fonctionnelles couvrant l'ensemble du cycle de vie d'un projet de construction : de la relation client à la facturation, en passant par la planification des tâches et le suivi contractuel. Les neuf exigences non-fonctionnelles fixent les contraintes de performance, de sécurité, d'ergonomie et de maintenabilité auxquelles la solution doit répondre.

Au niveau **architectural**, l'application repose sur une séparation nette en trois couches : interface React en SPA, API Laravel exposant des services REST sécurisés par Sanctum et RBAC, et persistance relationnelle via Eloquent. Le flux d'authentification, la matrice des permissions et l'organisation modulaire du code (backend par contrôleurs/services, frontend par features) garantissent une base solide pour l'évolution future de l'outil.

Au niveau **métier**, la conception détaillée des sept modules (projets, clients, documents commerciaux, contrats, tâches, tableau de bord, portail client) traduit les processus internes de CIVCO BTP en composants logiciels cohérents, avec des règles de gestion explicites et des flux documentaires tracés.

Enfin, la modélisation **UML** — cas d'utilisation, séquence, classes et activité — formalise les interactions, le modèle de données et les flux métier, et sert de référence pour la phase de réalisation décrite au chapitre suivant.

Le chapitre suivant décrit l'implémentation concrète de cette conception : choix d'environnement, structure du code, captures d'écran des modules principaux et résultats de la campagne de tests.

Chapitre 4 : Réalisation et Implémentation

4.1 Introduction

Ce chapitre décrit la réalisation concrète de l'application de gestion et de planification des projets BTP développée pour CIVCO BTP, à partir de la conception présentée au chapitre précédent. Nous présentons d'abord l'environnement technique et les outils utilisés, puis détaillons l'implémentation du backend Laravel, du frontend React et des principaux modules métier. Le chapitre se conclut par les tests réalisés et les résultats de validation.

4.2 Environnement technique

4.2.1 Stack technologique

Serveur applicatif PHP 8.3, Laravel 13.8, Laravel Sanctum 4 (authentification SPA), Eloquent ORM, migrations et seeders.

Interface utilisateur React 19.2, Vite 8, React Router 7, Tailwind CSS 4, Axios, Recharts (graphiques), Leaflet (cartographie).

Base de données MySQL / PostgreSQL (modèle relationnel, schéma versionné par migrations Laravel).

Outils complémentaires Git / GitHub (versionnement), Visual Studio Code (IDE), PHPUnit 12 (tests backend), ESLint (qualité frontend).

Le Tableau 4.1 récapitule les versions principales des composants utilisés en développement.

Tableau 4.1 : Versions des composants de la stack technique

Composant	Version	Rôle
PHP	8.3+	Langage backend, exécution Laravel
Laravel	13.8	Framework API, ORM, authentification
Laravel Sanctum	4.x	Sessions SPA sécurisées
React	19.2	Interface utilisateur composants
Vite	8.0	Build tool et serveur de développement
Tailwind CSS	4.3	Styles utilitaires
MySQL / PostgreSQL	8.x / 16	Persistance relationnelle
PHPUnit	12.5	Tests automatisés backend

4.2.2 Outils de développement

Gestion de projet Trello (tableau Kanban) pour planifier les modules, suivre l'avancement des tâches et préparer les démonstrations de suivi.

Contrôle de version Git et GitHub pour le suivi des évolutions, branches de fonctionnalités et revues de code.

IDE Visual Studio Code avec extensions PHP Intelephense, ESLint et Prettier.

Tests PHPUnit pour les tests d'intégration API ; tests manuels sur navigateur (Chrome, Edge) pour les parcours utilisateurs.

Qualité de code Laravel Pint (formatage PHP), ESLint (linting JavaScript/React).

Documentation API Routes Laravel exposées sous `/api/v1/`, testables via outils HTTP (Postman, curl) ou le navigateur pour les endpoints GET.

4.2.3 Architecture de déploiement

Environnement de développement

En développement, l'application repose sur deux processus parallèles, orchestrés depuis deux terminaux :

1. **Backend Laravel** : `php artisan serve --host=127.0.0.1 --port=8000` expose l'API sur le port 8000.
2. **Frontend Vite** : `npm run dev` lance le serveur de développement React sur le port 5173, avec rechargement à chaud (*hot-reload*).

Le fichier `vite.config.js` configure un proxy : les requêtes vers `/api`, `/sanctum` et `/storage` sont redirigées vers Laravel (port 8000), ce qui évite les problèmes de CORS en développement et simule un déploiement unifié.

Schéma de communication

- Le navigateur charge l'application React depuis `http://127.0.0.1:5173`.
- Les appels API transitent par le proxy Vite vers `http://127.0.0.1:8000/api/v1/`.
- Sanctum émet un cookie de session après authentification; Axios est configuré avec `withCredentials: true`.
- Les fichiers uploadés (logos, documents, signatures) sont stockés dans `storage/app/public` et servis via le lien symbolique `public/storage`.

Étude d'hébergement : OVHcloud vs o2switch

En vue d'une mise en production chez CIVCO BTP, une **étude comparative d'hébergement** a été menée entre deux prestataires français largement utilisés pour les applications web PHP : **OVHcloud** et **o2switch**. L'objectif était d'identifier la solution la mieux adaptée au profil de l'entreprise : application métier interne, équipe réduite, budget maîtrisé et besoin de simplicité d'administration.

Les critères d'évaluation retenus sont les suivants :

- **Compatibilité Laravel / PHP 8.3** : exécution de l'API backend et des commandes artisan (migrations, files d'attente, planificateur).
- **Base de données** : support MySQL ou PostgreSQL, sauvegardes et quotas.
- **Déploiement du frontend React** : hébergement des fichiers statiques produits par `npm run build`.
- **Sécurité** : certificat SSL, pare-feu, mises à jour système.
- **Coût et prévisibilité** : tarification mensuelle, absence de surprises sur la facturation.
- **Support et localisation** : assistance en français, datacenters en France, conformité pour des données clients BTP.
- **Facilité d'exploitation** : courbe d'apprentissage pour une PME sans équipe DevOps dédiée.

Tableau 4.2 : Comparatif d'hébergement : OVHcloud vs o2switch

Critère	OVHcloud	o2switch
Type d'offre	VPS, cloud public, serveurs dédiés (IaaS)	Hébergement mutualisé tout-en-un
Laravel / PHP	Compatible (configuration manuelle : Nginx, PHP-FPM, Composer)	Compatible nativement (PHP 8.x, SSH, Composer)
Base de données	MySQL/PostgreSQL à installer et administrer	MySQL inclus, bases multiples
Frontend React (build)	Fichiers statiques sur VPS ou Object Storage	Hébergement direct des assets dist/
SSL (HTTPS)	Let's Encrypt ou certificat payant (selon offre)	Let's Encrypt inclus et automatisé
Sauvegardes	À configurer (snapshots, scripts)	Sauvegardes journalières incluses
Tarification	Variable (facturation à l'usage, snapshots, trafic)	Forfait fixe illimité (offre unique)
Support	Tickets, documentation technique	Support français réactif, panel cPanel
Scalabilité	Élevée (montée en charge horizontale)	Modérée (mutualisé, adapté PME)
Complexité admin	Élevée (sysadmin requis)	Faible (panel graphique)

Analyse. OVHcloud [12] offre une grande flexibilité et une scalabilité adaptée aux applications à fort trafic ou aux architectures distribuées (conteneurs Docker, load balancing, bases managées). En revanche, le déploiement d'une application Laravel + React sur un VPS OVHcloud suppose une configuration manuelle du serveur web, de PHP-FPM, des certificats, des sauvegardes et de la supervision. Cette approche est pertinente pour des équipes disposant de compétences d'administration système, mais elle représente un surcoût en temps pour une PME comme CIVCO BTP.

o2switch [13], hébergeur français basé à Grabels, propose une offre mutualisée orientée simplicité : PHP récent, bases MySQL, certificats SSL gratuits, sauvegardes automatiques et support technique francophone. Pour une application métier interne au volume d'utilisateurs modéré (équipes CIVCO BTP et clients portail), cette formule couvre les besoins sans nécessiter la gestion d'un serveur virtuel. Le frontend React est déployé sous forme de fichiers statiques (`npm run build`), tandis que Laravel gère l'API et les sessions Sanctum sur le même domaine, simplifiant la configuration des cookies et du CSRF.

Choix retenu. Au terme de cette étude, **o2switch** a été retenu comme plateforme d'hébergement cible pour CIVCO BTP. Ce choix se justifie par :

- un **rapport qualité/prix** favorable pour une application interne ;

- une **mise en production plus rapide**, sans administration système avancée ;
- une **compatibilité native** avec la stack Laravel / MySQL ;
- un **support en français** et des datacenters localisés en France ;
- des **sauvegardes et SSL inclus**, répondant aux exigences de sécurité identifiées au Chapitre 3.

OVHcloud reste une alternative crédible si l'application devait évoluer vers une architecture plus complexe (montée en charge importante, microservices, conteneurisation Docker). Pour la première version de l'outil, la simplicité opérationnelle d'o2switch a été privilégiée.

Configuration : variables d'environnement

La configuration sensible est centralisée dans le fichier `.env` du backend. Le Tableau 4.3 liste les variables principales :

Tableau 4.3 : Variables d'environnement principales du fichier `.env`

Variable	Rôle
APP_URL	URL publique de l'application (ex. <code>http://127.0.0.1:8000</code>)
DB_CONNECTION / DB_DATABASE	Connexion SGBD (MySQL ou PostgreSQL)
SESSION_DRIVER	Pilote de session (fichier ou base de données)
SANCTUM_STATEFUL_DOMAINS	Domaines autorisés pour les cookies Sanctum
FRONTEND_URL	URL du frontend (pour les redirections et CORS)
MAIL_*	Configuration SMTP (notifications, invitations portail)

Initialisation de la base de données

Le démarrage d'un environnement neuf suit la séquence standard Laravel :

1. `composer install` : installation des dépendances PHP.
2. `cp .env.example .env` puis configuration de la base de données.
3. `php artisan key:generate` : génération de la clé d'application.
4. `php artisan migrate --seed` : création du schéma et données de démonstration.
5. `php artisan storage:link` : lien symbolique pour les fichiers publics.

Les seeders (DatabaseSeeder, DemoSeeder, RoleSeeder) peuplent la base avec des utilisateurs de test, des rôles, des permissions, des clients et des projets représentatifs du secteur BTP au Maroc.

4.3 Implémentation du backend

4.3.1 Structure du projet

Le backend suit l'architecture modulaire Laravel, organisée par responsabilité :

```
backend/
+-- app/
|   +-- Enums/                # Statuts (Project, Quote, Invoice, Client...)
|   +-- Http/
|       +-- Controllers/Api/V1/ # ~45 controleurs REST
|       +-- Middleware/         # Auth, permissions, contexte
|       +-- Requests/           # Validation par domaine
|       +-- Resources/          # Serialisation JSON
|   +-- Models/                 # Entites Eloquent (~35 modeles)
|   +-- Services/               # Logique metier (QuoteService, etc.)
+-- database/
|   +-- migrations/             # ~70 migrations versionnees
|   +-- seeders/                # Donnees de demo et referentiels
+-- routes/api.php              # Definition des routes /api/v1/
\-- tests/Feature/              # Tests PHPUnit d'integration
```

Cette organisation sépare clairement le transport HTTP (contrôleurs), la validation (Requets), la transformation des réponses (Resources) et la logique métier (Services), conformément au patron décrit au Chapitre 3.

4.3.2 API REST : endpoints principaux

L'API est regroupée sous le préfixe `/api/v1/`. Les endpoints principaux sont organisés par domaine fonctionnel :

Authentification POST `/login`, GET `/me`, POST `/logout`, PATCH `/me` (profil utilisateur).
Gestion de session via Sanctum.

Projets GET/POST `/projects`, GET/PATCH/DELETE `/projects/{id}`, gestion des phases, équipe, dépenses, médias et commentaires.

Clients GET/POST `/clients`, archivage, contacts, provisionnement portail (POST `/clients/{id}/portal-account`).

Devis GET/POST `/quotes`, lignes de détail, conversion en facture, suivi des statuts et impressions.

Factures GET/POST /invoices, paiements, état de facturation.

Bons de livraison GET/POST /delivery-forms, lignes et statuts.

Contrats GET/POST /contracts, compilation, signature, avenants (/contract-amendments).

Tâches GET/POST /tasks, assignation, filtrage par projet et utilisateur.

Équipe GET /team/members, gestion des membres et rôles.

Configuration /roles, /document-templates, /tenant/document-controls, thème et badges.

Tableau de bord GET /dashboard/summary, indicateurs agrégés.

Recherche GET /search, recherche globale multi-entités.

Portail client /client-portal/* : projets, devis, contrats, messagerie, acceptation et signature.

Historique GET /activity-logs, journal des actions.

Chaque route métier sensible est protégée par le middleware `permission:*`, qui vérifie que l'utilisateur authentifié dispose du droit requis avant d'exécuter l'action.

4.3.3 Services métier

La logique complexe est extraite des contrôleurs vers des classes Service :

- **QuoteService / InvoiceService** : calcul des totaux, numérotation, conversion devis → facture, gestion des statuts.
- **DeliveryFormService** : production des bons de livraison et liaison aux projets.
- **DocumentService / DocumentTemplateCompilationService** : génération et compilation des documents à partir de modèles.
- **DashboardService** : agrégation des indicateurs (projets actifs, tâches en retard, chiffre d'affaires).
- **AuthContextService** : résolution du contexte utilisateur (rôles, permissions, société courante).
- **ActivityLogService** : journalisation des actions sensibles.
- **PrintTrackingService** : suivi des impressions officielles et quotas.
- **SiteGeocodingService** : géolocalisation des chantiers (Nominatim + coordonnées de repli pour les villes marocaines).

4.3.4 Authentification et contrôle d'accès

L'authentification repose sur **Laravel Sanctum** en mode SPA :

1. Le frontend obtient un cookie CSRF via GET `/sanctum/csrf-cookie`.
2. L'utilisateur envoie ses identifiants à POST `/api/v1/login`.
3. Laravel valide, crée une session et retourne le profil (`UserResource`) avec rôles et permissions.
4. Chaque requête ultérieure inclut le cookie de session ; le middleware `auth:sanctum` protège les routes.
5. Le middleware `permission:slug` vérifie les droits granulaires (`project.view`, `quote.manage`, etc.).

Le service `PostLoginRedirectService` oriente l'utilisateur vers la page d'accueil adaptée à son profil (tableau de bord admin, liste des projets pour un chef de projet, factures pour un comptable).

4.3.5 Migrations et modèle de données

Le schéma de base compte environ 70 migrations, couvrant l'ensemble des entités métier. Les migrations sont idempotentes et versionnées : chaque modification de schéma produit un nouveau fichier daté, exécutable de façon reproductible sur tout environnement.

Les relations Eloquent principales ont été implémentées comme suit :

- Client hasMany Project
- Project belongsTo Client, hasMany ProjectPhase, Task, Quote, Contract
- Quote hasMany QuoteLine, hasOne Invoice
- User belongsToMany Project (équipe), belongsTo Role

Les **API Resources** (`ProjectResource`, `QuoteResource`, etc.) contrôlent les champs exposés au frontend et permettent d'inclure conditionnellement les relations (`client`, `lines`, `teamMembers`).

4.3.6 Middleware et sécurité des routes

Le pipeline de middleware Laravel applique une chaîne de vérifications sur chaque requête API protégée :

1. **auth :sanctum** : vérifie que l'utilisateur est authentifié via session.
2. **EnsureCompanyContext** : résout la société courante à partir de l'en-tête X-Company-Id ou du contexte utilisateur.
3. **permission :slug** : contrôle granulaire par action métier (`client.view`, `invoice.manage`, etc.).
4. **ValidateCsrfToken** : protection contre les requêtes forgées sur les verbes mutants (POST, PUT, PATCH, DELETE).

Cette approche en profondeur (*defense in depth*) garantit qu'aucune action métier ne peut être exécutée sans authentification et sans autorisation explicite, conformément aux bonnes pratiques de sécurité web [14].

4.3.7 Exemple : création d'un devis

Le flux de création d'un devis illustre l'interaction entre les couches :

1. Le frontend envoie POST `/api/v1/quotes` avec le corps JSON (client, projet, lignes, montants).
2. `StoreQuoteRequest` valide les champs (références obligatoires, montants positifs, dates cohérentes).
3. `QuoteController@store` délègue à `QuoteService::create()`.
4. Le service crée l'enregistrement `Quote` et ses `QuoteLine` associées dans une transaction base de données.
5. Les totaux HT, TVA et TTC sont calculés automatiquement.
6. `QuoteResource` sérialise la réponse JSON pour le frontend.
7. `ActivityLogService` journalise l'action si configuré.

Ce patron (Request → Controller → Service → Resource) est reproduit sur l'ensemble des modules métier.

4.3.8 Modules complémentaires

En complément des modules principaux, plusieurs fonctionnalités transverses ont été implémentées :

Notifications : le modèle Notification et le contrôleur associé permettent d’informer les utilisateurs des événements importants (nouvelle tâche assignée, devis accepté, message reçu). L’interface affiche un badge de notification non lue dans l’en-tête.

Messagerie : le module PortalMessage gère les échanges entre l’équipe interne et les clients externes, scopés par projet. Le service PortalMessageService filtre les conversations selon les droits de l’utilisateur (`can_chat_with_client` sur les membres d’équipe).

Recherche globale : le composant GlobalSearch et l’endpoint GET `/search` permettent de retrouver rapidement un client, un projet, un devis ou une facture par mot-clé, améliorant la productivité au quotidien.

Contrôle d’impression : PrintTrackingService gère les quotas d’impressions officielles par type de document et par en-tête (avec ou sans papier à en-tête). Au-delà du quota, les impressions sont marquées «COPIE - NON OFFICIEL».

Journal d’activité : chaque action sensible (création de projet, validation de devis, archivage client) est enregistrée par ActivityLogService avec l’identité de l’utilisateur, le type d’action et l’horodatage. Le module `features/history/` expose ces entrées sous forme de fil d’audit consultable par l’administrateur.

Géocodage des chantiers : lors de la saisie d’une adresse de chantier, SiteGeocodingService tente de résoudre automatiquement les coordonnées GPS via l’API Nominatim (OpenStreetMap). En cas d’échec, un référentiel de coordonnées des principales villes marocaines (MoroccoCityCoordinates) assure un positionnement approximatif sur la carte Leaflet.

Modèles de documents : le module DocumentTemplate permet de définir des gabarits de contrats et de documents commerciaux avec des champs dynamiques (placeholders). Le service DocumentTemplateCompilationService injecte les données du projet et du client lors de la génération du document final.

Gestion d’équipe : TeamMemberService et TeamQueryService centralisent l’affectation des collaborateurs aux projets, la gestion des rôles sur chantier et le contrôle de l’autorisation de messagerie client (`can_chat_with_client`).

4.4 Implémentation du frontend

4.4.1 Architecture React

Le frontend est une SPA React 19 organisée par domaines métier (*features*) :

```
frontend/src/
+-- api/                # Modules Axios par ressource
```

```

|   +-- client.js           # Instance Axios + intercepteurs
|   +-- projects.js, clients.js, quotes.js, invoices.js...
+-- components/            # Composants transverses (Sidebar, Modal...)
+-- context/
|   +-- AuthContext.jsx     # Session, roles, permissions
|   +-- ThemeContext.jsx    # Theme sombre, couleurs societe
|   +-- LanguageContext.jsx # i18n FR / EN
+-- features/
|   +-- dashboard/          # Tableau de bord et widgets
|   +-- projects/           # Liste, detail, phases, equipe
|   +-- clients/            # Gestion clients et portail
|   +-- quotes/             # Devis
|   +-- invoices/           # Factures
|   +-- delivery-forms/     # Bons de livraison
|   +-- tasks/              # Taches
|   +-- configuration/      # Parametres, roles, modeles
|   +-- profile/            # Profil utilisateur
+-- routes/
|   +-- AppRoutes.jsx       # Definition des routes
|   +-- routePermissions.js # Controle d'accès par route
\-- i18n/locales/          # Traductions fr.js, en.js

```

4.4.2 Client HTTP et authentification

Le module `api/client.js` centralise la configuration Axios :

- `baseUrl` pointant vers `/api/v1` (proxifié par Vite).
- `withCredentials: true` pour transmettre les cookies Sanctum.
- Intercepteur de requête : récupération du jeton CSRF depuis le cookie `XSRF-TOKEN` et injection dans l'en-tête `X-XSRF-TOKEN`.
- Intercepteur de réponse : redirection vers `/login` en cas de `401 Unauthorized`.
- En-tête `X-Company-Id` pour le contexte société lorsque requis.

Le `AuthContext` expose l'état global d'authentification : utilisateur courant, rôles, permissions, fonctions `login / logout`, et helpers de vérification des droits consommés par les composants `PermissionGate` et `AdminRoute`.

4.4.3 Routage et contrôle d'accès

`AppRoutes.jsx` définit l'arborescence de navigation :

- Routes publiques : `/login`.
- Routes authentifiées : `/` (dashboard), `/projects`, `/clients`, `/quotes`, `/invoices`, `/tasks`, `/map`, `/configuration`, `/history`.
- Portail client : `/portal/*` (interface dédiée aux clients externes).
- Protection : composants `ProtectedRoute`, `AdminRoute` et fonction `canAccessRoute()` qui croise le chemin demandé avec les permissions de l'utilisateur.

4.4.4 Composants transverses

Plusieurs composants partagés structurent l'expérience utilisateur :

- **Sidebar** : navigation principale, filtrée selon les permissions.
- **AppLayout** : enveloppe commune (sidebar + en-tête + contenu).
- **GlobalSearch** : recherche unifiée clients, projets, documents.
- **CommercialPrintSheet** : génération et impression des documents commerciaux (devis, factures, BL) avec suivi des impressions.
- **PrintOptionsModal** : choix d'impression (avec/sans en-tête, copie officielle).

4.4.5 Internationalisation

L'application supporte le français (par défaut) et l'anglais via `LanguageContext` et les fichiers `i18n/locales/fr.js` et `en.js`. Les libellés de l'interface, les messages d'erreur et les intitulés de menus sont externalisés, facilitant l'ajout de nouvelles langues.

4.4.6 Détail des modules frontend

Chaque module *feature* suit une structure cohérente : une page liste (`*Page.jsx`), une ou plusieurs pages détail (`*DetailPage.jsx`), des composants locaux (`components/`) et un module API dédié (`api/*.js`).

Module Projets (`features/projects/`) : `ProjectsPage` affiche la liste filtrable des chantiers ; `ProjectDetailPage` regroupe les onglets phases, tâches, équipe, documents et contrats. Le composant `ProjectAmendmentsTab` gère les avenants contractuels.

Module Clients (features/clients/) : ClientsPage avec recherche et filtres; NewClientModal pour la création; ClientPortalPanel pour la gestion de l'accès portail. Le composant ConfirmArchiveModal sécurise l'archivage.

Module Documents commerciaux : QuotesPage, QuoteDetailPage, InvoicesPage, DeliveryFormDetailPage. Les composants d'impression (CommercialPrintSheet, DeliveryFormPrintSheet) génèrent les versions PDF imprimables.

Module Configuration (features/configuration/) : panneaux dédiés pour les rôles (RolesSettingsPanel), les modèles de contrats (ContractTemplateSettingsPanel), les contrôles d'impression (DocumentControlsSettingsPanel) et les modèles de documents (DocumentTemplatesSettingsPanel).

4.4.7 Gestion d'état et hooks

L'état applicatif repose principalement sur React Context :

- **AuthContext** : session utilisateur, permissions, fonctions login/logout.
- **ThemeContext** : couleurs de la société injectées en variables CSS.
- **LanguageContext** : bascule FR / EN de l'interface.
- **usePolicyPrint / useProjectTasks** : hooks métier encapsulant la logique d'impression et de chargement des tâches projet.

React Query (@tanstack/react-query) est installé et prêt pour le cache des requêtes API; son adoption progressive permettra d'optimiser les rechargements de listes et fiches détail.

4.5 Déroulement du développement et livrables

4.5.1 Phases de réalisation

Le développement a suivi la planification définie au Chapitre 1, avec une progression par modules fonctionnels. Chaque phase a produit un incrément démontrable lors des réunions de suivi hebdomadaires avec l'encadrant professionnel.

Tableau 4.4 : Phases de développement et modules livrés

Phase	Modules livrés	Validation
Fondations	Auth Sanctum, RBAC, navigation React, layout	Connexion multi-profils
Cœur métier	Projets, clients, phases, équipe	CRUD complet, carte
Documents	Devis, factures, BL, impression	Cycle commercial bout en bout
Opérations	Tâches, contrats, avenants, dépenses	Suivi opérationnel
Collaboration	Portail client, messagerie, notifications	Parcours client externe
Gouvernance	Configuration, historique, recherche globale	Administration complète

4.5.2 Synthèse quantitative des livrables

Au terme du stage, l'application comprend les éléments quantitatifs suivants, illustrant l'ampleur de la réalisation :

- Plus de **70 migrations** de base de données versionnées.
- Environ **40 contrôleurs API** et **25 services métier**.
- Plus de **30 modules frontend** (*features* et composants).
- Une suite de **7 classes de tests** PHPUnit couvrant les flux critiques.
- Plus de **200 routes API** protégées par middleware d'authentification et de permissions.
- Support **bilingue** (français / anglais) sur l'ensemble de l'interface.

4.5.3 Bonnes pratiques adoptées

Plusieurs bonnes pratiques de génie logiciel ont été appliquées tout au long du développement :

- **Séparation des responsabilités** : contrôleurs fins, logique métier dans les services, validation dans les Form Requests.
- **Conventions de nommage** : respect des standards Laravel (PascalCase pour les classes, snake_case pour les colonnes) et React (PascalCase pour les composants).
- **Versionnement Git** : branches par fonctionnalité, commits atomiques, messages descriptifs.

- **Suivi Kanban (Trello)** : chaque module ou correction était modélisé sous forme de carte, déplacée au fil de l'avancement entre les colonnes du tableau.
- **Revue de code** : validation des choix techniques avec l'encadrant professionnel avant intégration des modules sensibles (facturation, permissions).
- **Documentation inline** : commentaires sur les règles métier non évidentes, docblocks sur les méthodes de service exposées à l'API.

4.6 Interfaces principales

Cette section présente les écrans clés de l'application. Les captures d'écran seront insérées ultérieurement ; les descriptions ci-dessous reflètent l'implémentation actuelle.

4.6.1 Page de connexion

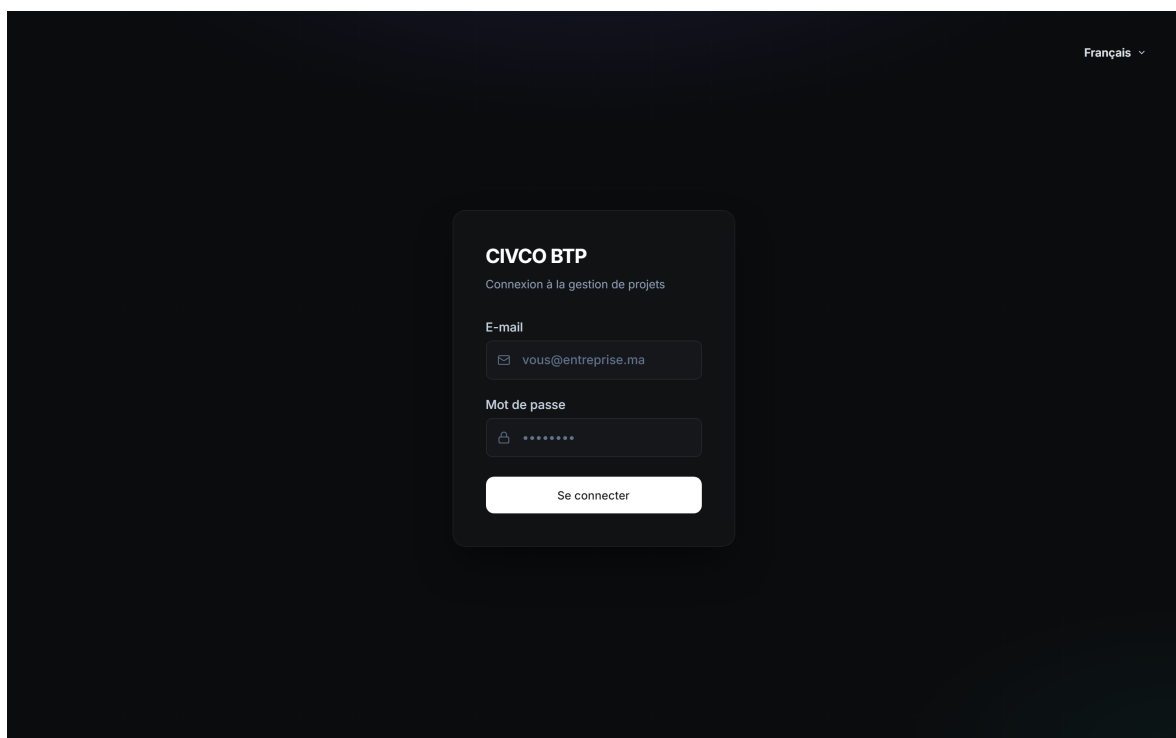


Figure 4.1 : Page de connexion : authentification par email et mot de passe

La page de connexion (Figure 4.1) permet à tout utilisateur interne de CIVCO BTP de s'authentifier. Après validation, l'utilisateur est redirigé vers la page d'accueil correspondant à son profil métier.

L'écran affiche le logo de l'entreprise, un formulaire email/mot de passe et une option « Se souvenir de moi ». Les erreurs d'authentification (identifiants incorrects, compte désactivé)

sont affichées de manière explicite. Le design sombre et épuré a été choisi pour réduire la fatigue visuelle lors d'une utilisation quotidienne prolongée.

4.6.2 Tableau de bord

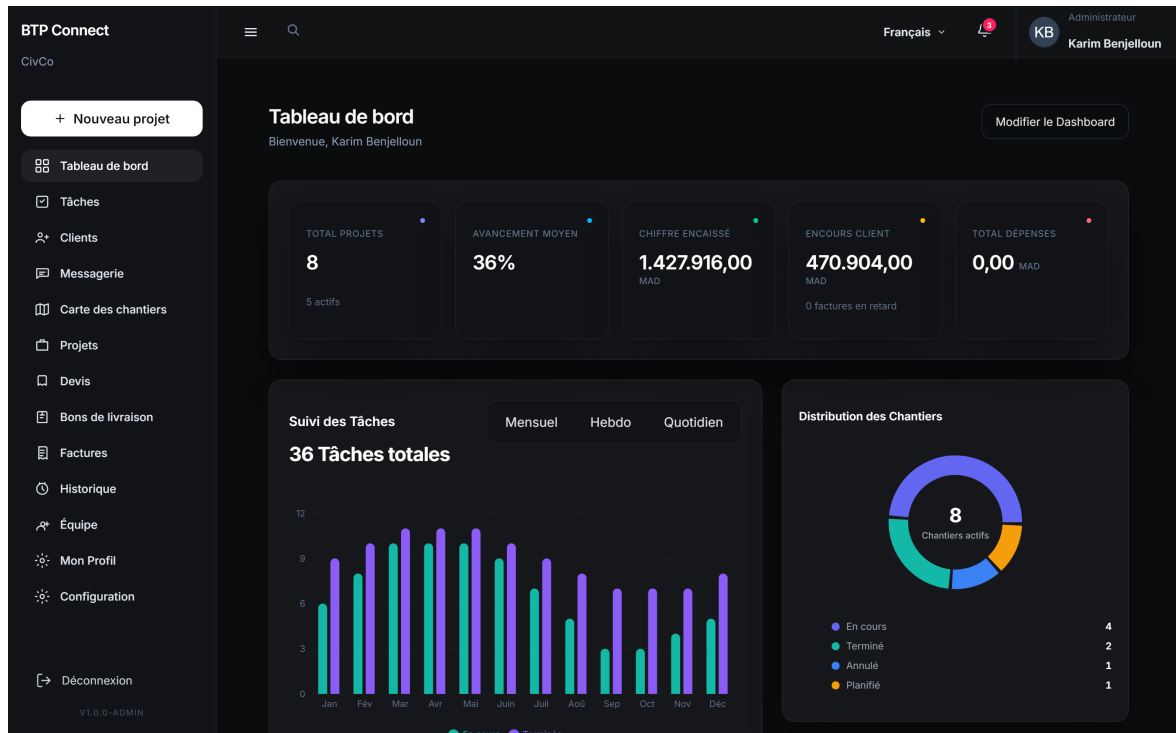


Figure 4.2 : Tableau de bord : vue synthétique de l'activité (projets, tâches, indicateurs)

Le tableau de bord (Figure 4.2) agrège les informations essentielles : nombre de projets en cours, tâches prioritaires, graphiques d'avancement (Recharts) et accès rapide aux modules. Les widgets affichés dépendent des permissions de l'utilisateur connecté.

Les widgets sont configurés dynamiquement : un administrateur voit l'ensemble des indicateurs (projets actifs, factures en attente, tâches en retard), tandis qu'un technicien ne voit que ses tâches assignées et les projets auxquels il participe. Les graphiques Recharts illustrent l'évolution de l'avancement et la répartition des projets par statut.

4.6.3 Gestion des projets

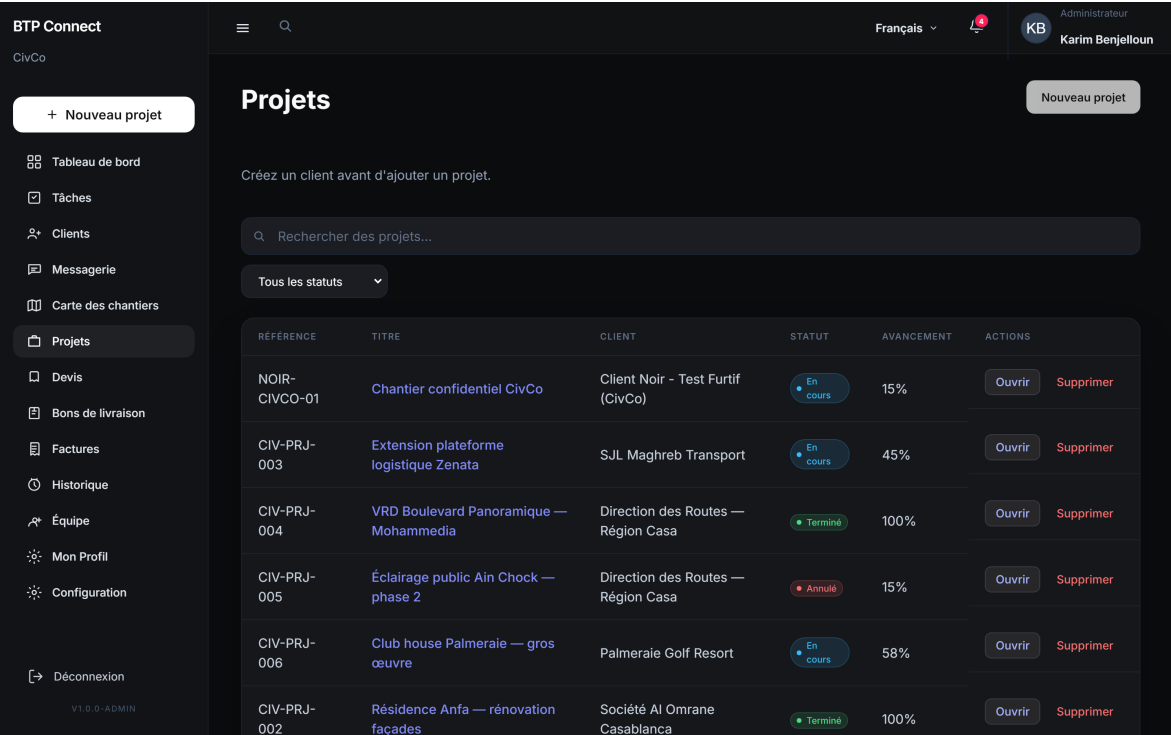


Figure 4.3 : Module Projets : liste des chantiers et fiche détail (phases, équipe, documents)

Le module Projets (Figure 4.3) permet de créer, consulter et modifier les chantiers. La fiche détail est organisée en onglets : informations générales, phases, tâches, équipe, devis/factures liés, contrats, dépenses et médias.

La liste des projets propose des filtres par statut, client et nature de chantier. La fiche détail centralise toutes les informations d’un chantier : depuis un seul écran, le chef de projet peut consulter l’avancement des phases, réaffecter un collaborateur, joindre un document ou créer un devis lié au projet.

4.6.4 Gestion des clients

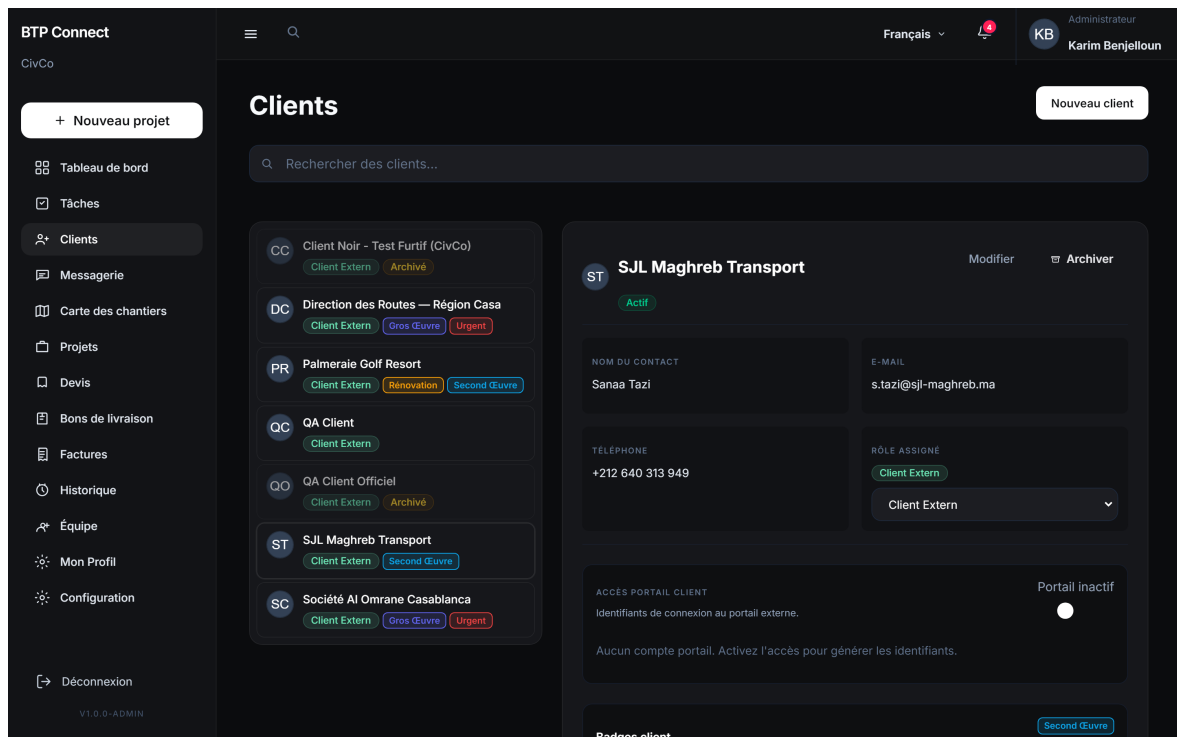


Figure 4.4 : Module Clients : fiches clients, contacts et archivage

Le module Clients (Figure 4.4) centralise les maîtres d'ouvrage et clients commerciaux. L'administrateur peut provisionner un accès portail pour un client externe directement depuis la fiche client.

Chaque fiche client regroupe les coordonnées, les contacts associés, les badges métier (Gros Œuvre, Urgent, etc.), l'historique des projets et l'état du compte portail. L'archivage est sécurisé par une modale de confirmation pour éviter les suppressions accidentelles.

4.6.5 Documents commerciaux

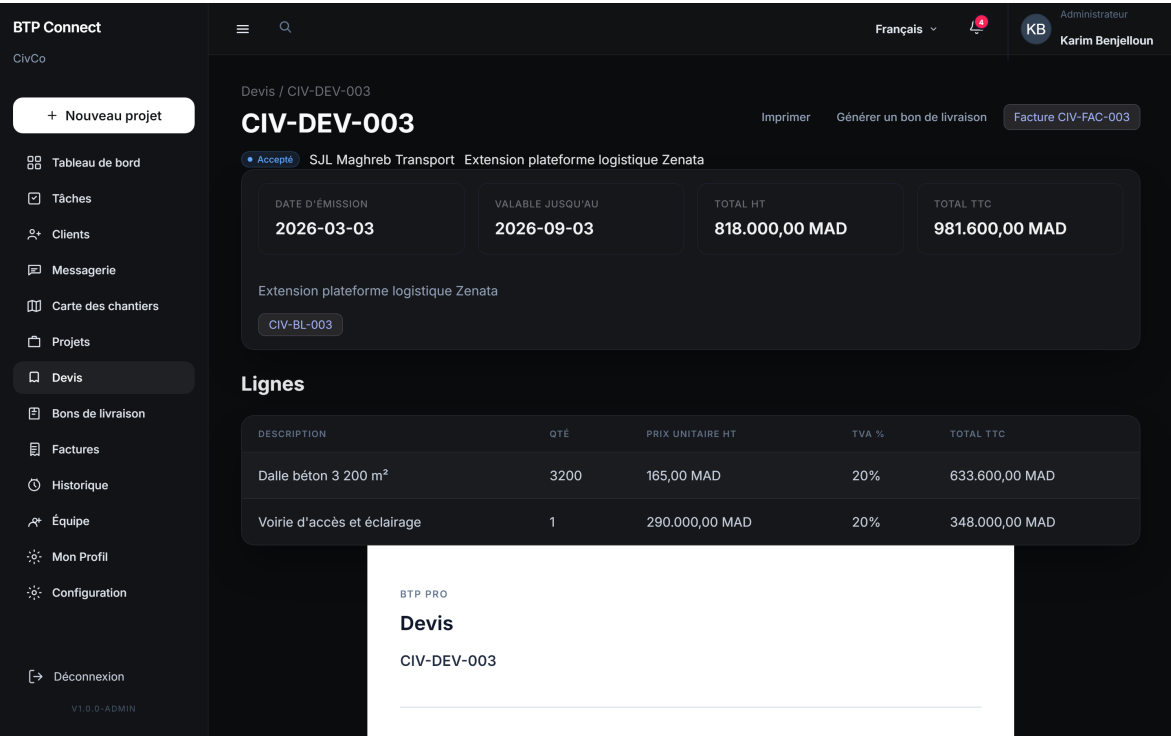


Figure 4.5 : Module Devis : création avec lignes de détail et calcul automatique des totaux

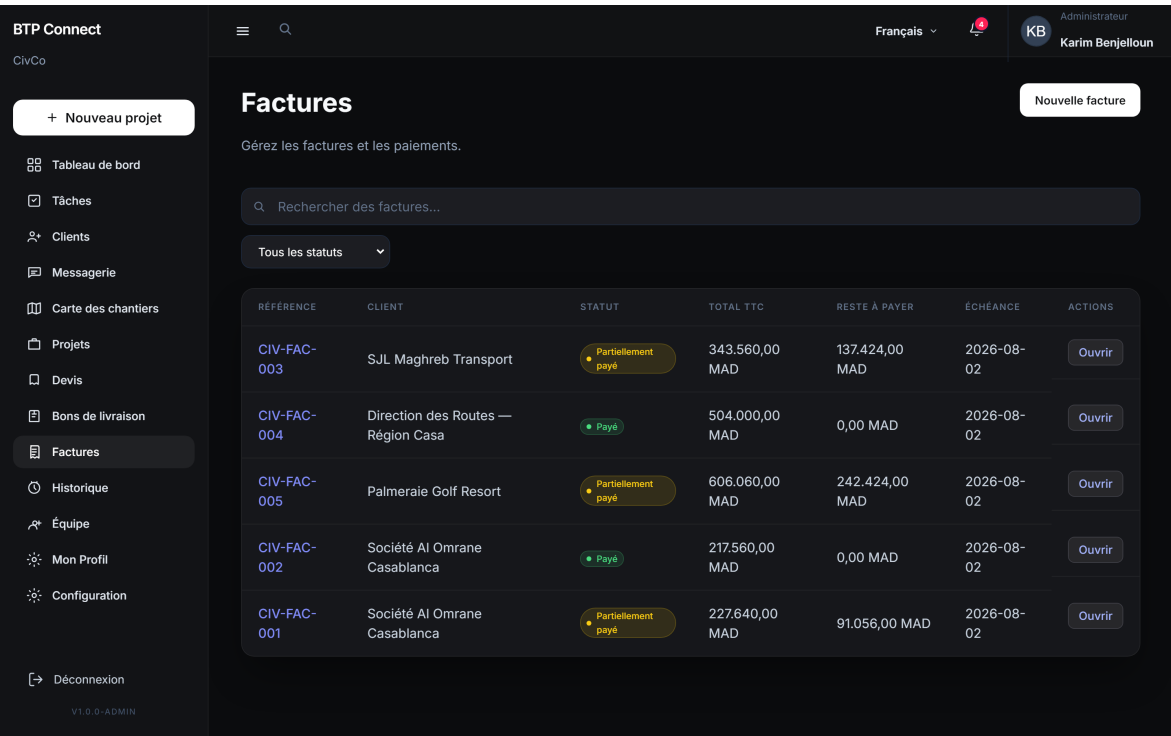


Figure 4.6 : Module Factures : suivi des factures et état de paiement

Les modules Devis et Factures (Figures 4.5 et 4.6) permettent la production des documents commerciaux, leur impression (avec suivi des copies officielles) et la conversion devis

→ facture en un clic.

Le formulaire de création de devis permet d'ajouter dynamiquement des lignes de prestation (description, quantité, prix unitaire HT, taux de TVA). Les totaux sont recalculés en temps réel. L'impression s'effectue via un composant React dédié (CommercialPrintSheet) qui génère une version formatée prête à l'envoi au client, avec suivi du quota d'impressions officielles.

4.6.6 Planification des tâches

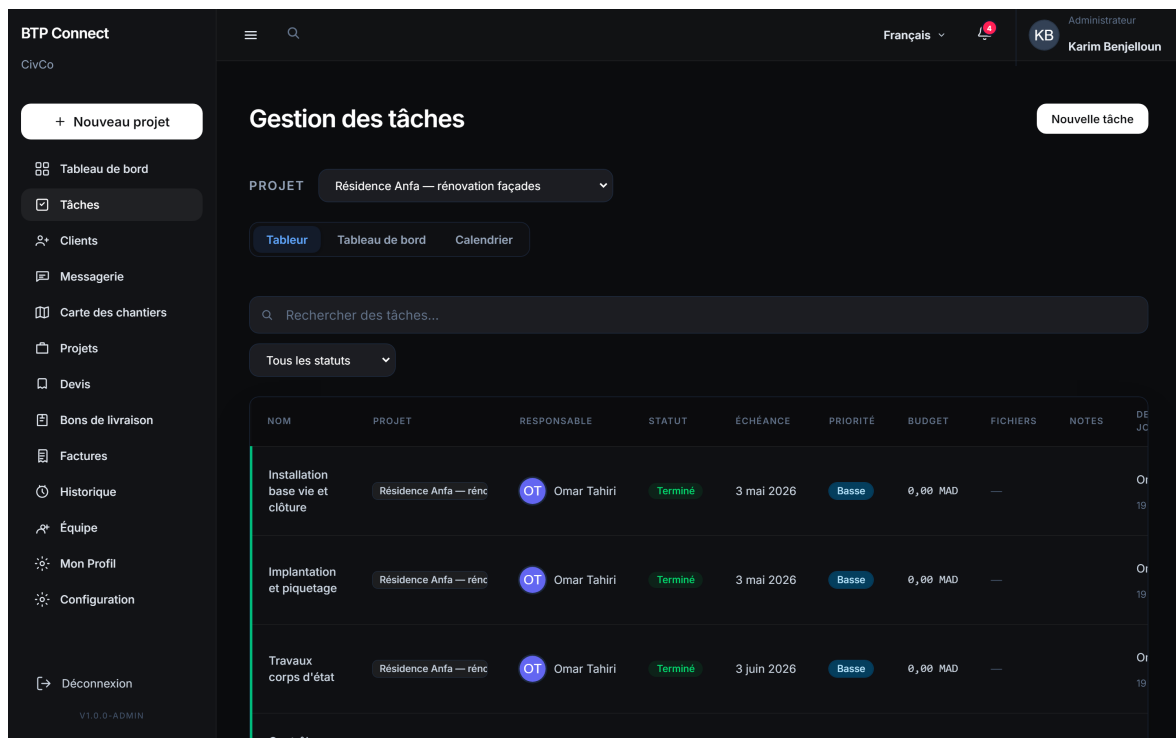


Figure 4.7 : Module Tâches : planification, assignation et suivi par projet

Le module Tâches (Figure 4.7) offre une vue tableau des missions opérationnelles, filtrable par statut, projet et assigné. Les chefs de projet peuvent créer et assigner des tâches ; les utilisateurs internes consultent leurs tâches propres.

Le module propose une vue tableau avec tri par date d'échéance, filtre par statut (à faire, en cours, terminé) et code couleur pour les tâches en retard. La création d'une tâche associe obligatoirement un projet, une phase et un responsable, garantissant la traçabilité de l'assignation.

4.6.7 Carte des chantiers

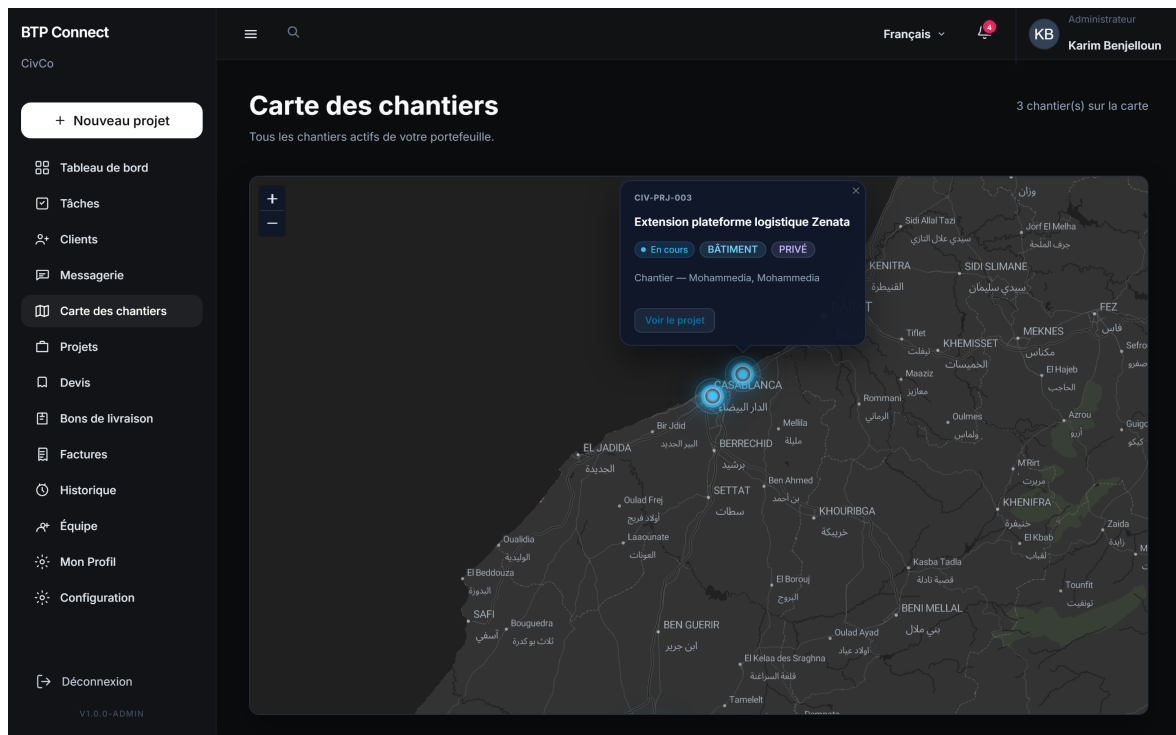


Figure 4.8 : Carte des projets : visualisation géographique des chantiers (Leaflet)

La carte (Figure 4.8) affiche les projets géolocalisés sur une carte interactive Leaflet, utile pour visualiser la répartition géographique des chantiers de CIVCO BTP.

Chaque marqueur sur la carte correspond à un projet géolocalisé. Un clic affiche un résumé (référence, client, statut, avancement) et un lien vers la fiche détail. Cette visualisation est particulièrement utile pour la direction et les chefs de projet qui gèrent plusieurs chantiers répartis sur différentes zones géographiques du Grand Casablanca et au-delà.

4.6.8 Portail client

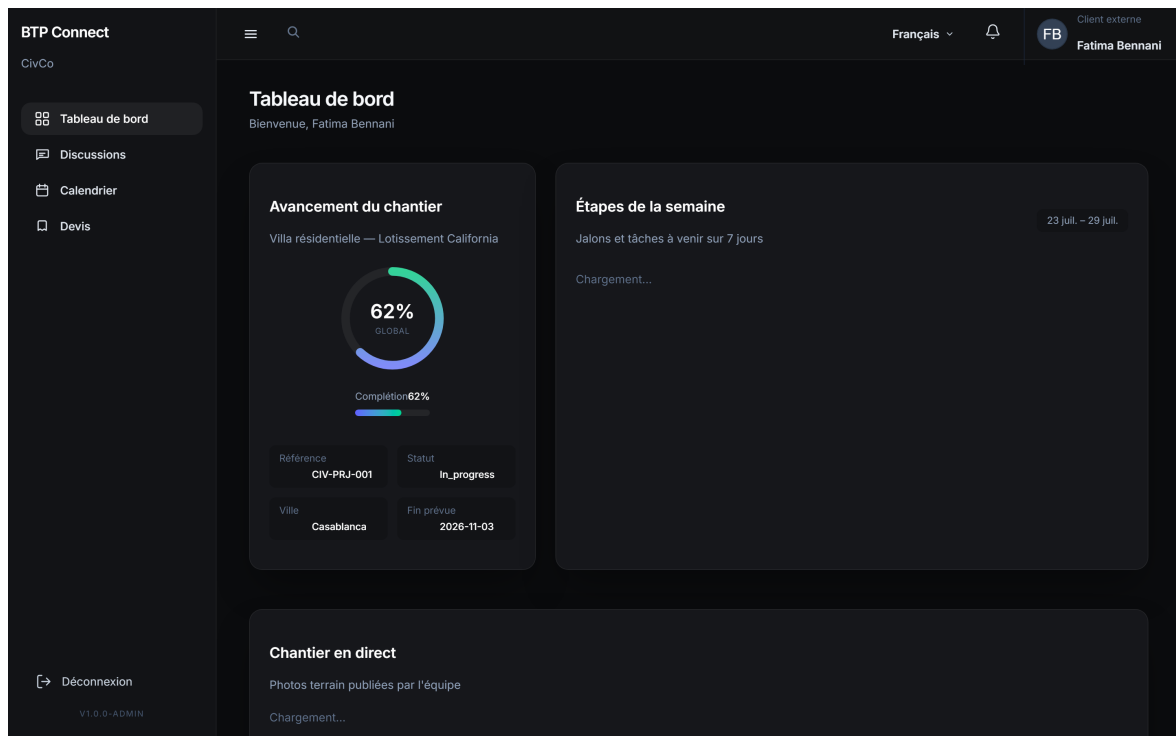


Figure 4.9 : Portail client : consultation des projets, acceptation de devis et signature

Le portail client (Figure 4.9) offre une interface simplifiée aux clients externes : consultation de l'avancement, acceptation de devis, signature de contrats et messagerie avec l'équipe projet.

L'interface portail est volontairement distincte de l'ERP interne : navigation réduite à quatre entrées (tableau de bord, discussions, calendrier, devis), sans accès aux modules de gestion internes (projets, facturation, configuration). Les routes React sont regroupées sous le préfixe `/portal/*` et protégées par le middleware d'authentification ; un utilisateur possédant le rôle `client_extern` est automatiquement redirigé vers le portail après connexion, tandis qu'un collaborateur interne ne peut pas y accéder.

Tableau de bord client. L'écran principal (Figure 4.9) présente trois blocs d'information. Le widget *Avancement du chantier* affiche un indicateur circulaire (ProgressRing) alimenté par le champ `progress_percent` du projet, accompagné de la référence chantier, du statut, de la ville et de la date de fin prévue. Le panneau *Étapes de la semaine* liste les jalons et tâches à échéance sur les sept prochains jours, filtrés par projet sélectionné. Enfin, le fil *Chantier en direct* (ChantierLiveFeed) expose les photos terrain publiées par l'équipe CIVCO BTP, rafraîchies périodiquement pour offrir une visibilité continue sur l'avancement des travaux.

Parcours documentaire et contractuel. Depuis l'onglet *Devis*, le client consulte les propositions commerciales qui le concernent et peut accepter un devis en un clic ; le statut est immédiatement mis à jour côté serveur. Pour les contrats, le composant

ClientContractSignature intègre un pad de signature tactile (canvas HTML5) : la signature est capturée, transmise à l'API et associée au document contractuel, déclenchant la mise à jour du statut et la notification de l'équipe projet.

Provisionnement et sécurité. L'accès portail est créé par l'administrateur depuis la fiche client (module Clients, Chapitre 4) : un identifiant de connexion est généré, associé au client concerné via le champ `client_id` de l'utilisateur, et activé ou désactivé à la demande. Toutes les requêtes API du portail sont filtrées par ce lien client → projet : un client externe ne voit que ses propres chantiers, devis et échanges, conformément au principe de moindre privilège défini au Chapitre 3.

4.6.9 Configuration

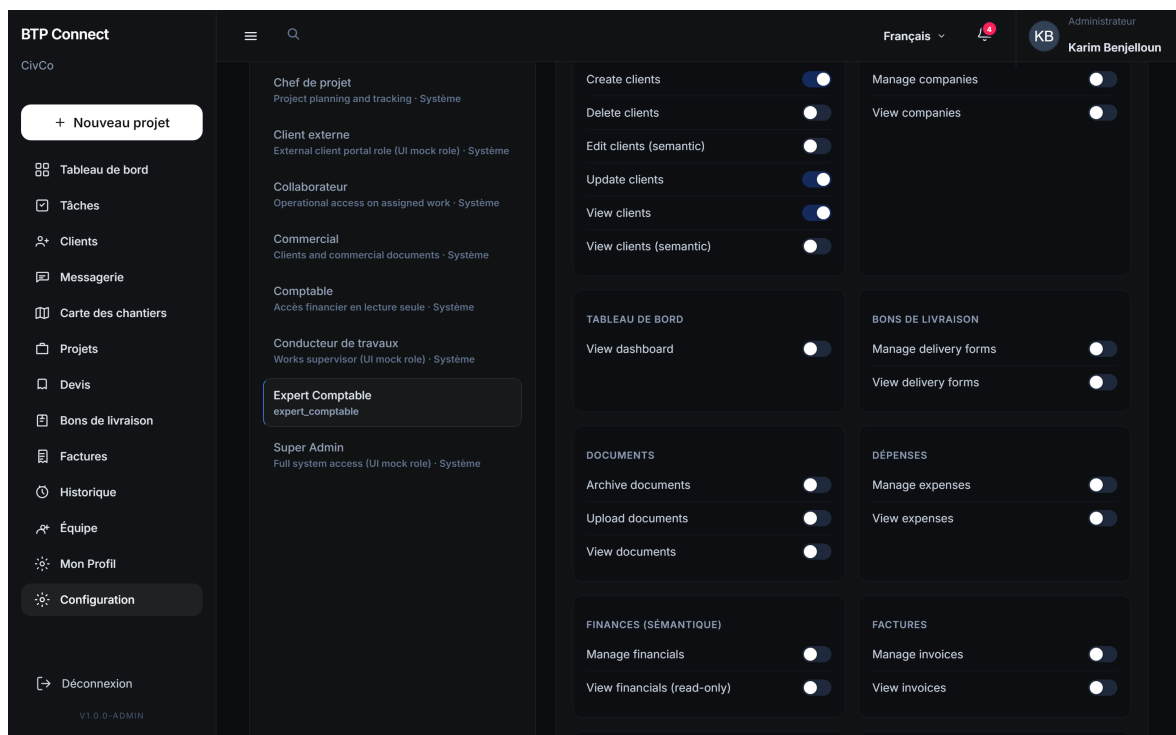


Figure 4.10 : Module Configuration : rôles, modèles de documents, contrôles d'impression

Le module Configuration (Figure 4.10) est réservé à l'administrateur : gestion des rôles et permissions, modèles de contrats et documents, paramètres d'impression, thème visuel et référentiels (badges, lots, secteurs).

Le panneau de configuration est organisé en onglets thématiques : rôles et permissions, modèles de contrats, contrôles d'impression, modèles de documents et paramètres généraux. L'administrateur peut créer un nouveau rôle métier, lui attribuer un sous-ensemble de permissions et l'assigner aux utilisateurs concernés, sans intervention technique sur le code.

4.7 Tests et validation

La validation de l'application s'appuie sur deux dispositifs complémentaires : une **suite de tests automatisés** PHPUnit au niveau du backend, et une **campagne de tests manuels** couvrant les parcours utilisateurs de bout en bout.

4.7.1 Tests automatisés backend

Le projet inclut une suite de tests d'intégration PHPUnit dans `tests/Feature/` :

AuthTest Connexion, déconnexion, accès au profil utilisateur, rejet des identifiants invalides.

ClientProjectTest CRUD clients et projets, relations, contraintes d'accès.

QuoteInvoiceTest Création de devis, lignes de détail, conversion en facture, calcul des totaux.

DocumentTest Upload et gestion des documents attachés aux projets.

ExpenseTest Enregistrement des dépenses de chantier.

DashboardTest Agrégation des indicateurs du tableau de bord.

RoleAccessTest Vérification des redirections post-login et des permissions par profil (comptable, commercial, chef de projet, administrateur).

Les tests utilisent `RefreshDatabase` et les seeders de démonstration pour garantir un environnement isolé et reproductible. La commande `php artisan test` exécute l'ensemble de la suite.

4.7.2 Campagne de tests manuels

Une checklist de tests manuels a été établie pour valider les parcours que les tests automatisés ne couvrent pas entièrement :

- **Authentification** : connexion, déconnexion, expiration de session, redirection selon le profil.
- **Projets** : création complète (client, phases, équipe, géolocalisation), modification, consultation par un utilisateur non-admin.
- **Documents commerciaux** : cycle devis → acceptation → facture → paiement ; impression avec et sans en-tête.

- **Contrats** : compilation depuis un modèle, signature portail client.
- **Tâches** : création, assignation, changement de statut, visibilité selon les permissions.
- **Portail client** : connexion client, consultation projet, acceptation devis, messagerie.
- **Configuration** : création de rôle, attribution de permissions, modification du thème.
- **Responsive** : consultation sur poste de bureau et tablette.

4.7.3 Validation qualitative

L'application a été validée en conditions réelles au sein de CIVCO BTP, avec les observations suivantes :

- **Adéquation métier** : les modules couvrent les processus quotidiens identifiés au Chapitre 1 (suivi de chantier, production documentaire, planification).
- **Ergonomie** : l'interface sombre et la navigation par sidebar ont été appréciées par les testeurs internes pour un usage prolongé.
- **Performance** : les listes et fiches détail se chargent en moins de 500 ms en environnement local ; le tableau de bord agrège les indicateurs sans latence perceptible.
- **Sécurité** : les tests de contournement de permissions (accès direct à une URL sans droit) sont correctement bloqués côté API avec une réponse 403 Forbidden.
- **Impression** : le mécanisme de suivi des copies officielles et le filigrane « COPIE - NON OFFICIEL » fonctionnent conformément aux exigences métier.

4.7.4 Synthèse des tests

Le Tableau 4.5 récapitule la couverture de validation par module.

Tableau 4.5 : Synthèse de la couverture de tests par module

Module	Auto.	Manuel	Points vérifiés
Authentification	✓	✓	Login, logout, session, redirection
Projets	✓	✓	CRUD, phases, équipe, carte
Clients	✓	✓	CRUD, archivage, portail
Devis / Factures	✓	✓	Lignes, conversion, impression
Tâches	—	✓	Assignation, statuts, permissions
Contrats	—	✓	Compilation, signature, avenants
Configuration	—	✓	Rôles, modèles, thème
Portail client	—	✓	Consultation, acceptation, messagerie

4.8 Difficultés rencontrées et solutions

4.8.1 Gestion des permissions granulaires

La mise en place du **RBAC** avec plus de trente permissions a nécessité une coordination entre le backend (middleware, seeders) et le frontend (PermissionGate, routePermissions). La solution retenue centralise la résolution des permissions dans AuthContextService côté API et permissionResolver.js côté React, garantissant une source de vérité unique.

4.8.2 Impression documentaire

La génération de PDF côté client (composants React dédiés) et le suivi des quotas d'impression officielle ont demandé un mécanisme de comptage persistant (PrintTrackingService) et une interface de choix avant chaque impression (PrintOptionsModal).

4.8.3 Géolocalisation des chantiers

L'intégration de la carte Leaflet avec géocodage automatique des adresses de chantier a été réalisée via SiteGeocodingService, qui interroge l'API Nominatim avec un repli sur un référentiel de coordonnées des villes marocaines (MoroccoCityCoordinates) en cas d'échec.

4.8.4 Synchronisation frontend / backend

La cohérence entre les permissions affichées dans l'interface et celles vérifiées côté API a nécessité un travail de synchronisation : chaque route frontend dans routePermissions.js doit correspondre à un contrôle permission:* sur l'endpoint associé. Les tests RoleAccessTest et les tests manuels multi-profils ont permis de détecter et corriger les écarts.

Conclusion

Ce chapitre a présenté la réalisation concrète de l'application de gestion de projets BTP développée pour CIVCO BTP. Au-delà de la conception formalisée au chapitre précédent, nous avons décrit l'environnement d'exécution, l'organisation du code, les modules livrés et les dispositifs de validation retenus pour garantir la fiabilité de la solution en conditions d'usage réelles.

Environnement et architecture. L'application repose sur une stack **Laravel 13 / React 19 / Vite / MySQL**, orchestrée en développement par deux processus parallèles (php

artisan serve et `npm run dev`). Le proxy Vite assure la communication transparente entre la SPA et l'API, tandis que **Laravel Sanctum** gère l'authentification par session sécurisée. Cette séparation frontend / backend a permis de maintenir une frontière nette : l'interface React ne contient aucune règle métier critique ; la validation, les calculs financiers et la persistance sont systématiquement assurés côté serveur.

Implémentation backend. L'API REST expose plus de quarante contrôleurs sous le préfixe `/api/v1/`, structurés par domaine (projets, clients, devis, factures, tâches, contrats, configuration). Chaque endpoint sensible est protégé par un middleware de permission (`permission:*`) et validé par un Form Request dédié. Les services métier (`PrintTrackingService`, `DocumentTemplateCompilationService`, `TeamMemberService`, etc.) centralisent les règles de gestion, tandis que les API Resources normalisent les réponses JSON consommées par le frontend.

Interface utilisateur. Le frontend React est organisé en modules par domaine fonctionnel, avec une sidebar filtrée selon le profil connecté. Les écrans principaux ont été implémentés et illustrés dans ce chapitre : authentification (Figure 4.1), tableau de bord (Figure 4.2), gestion des projets et des clients (Figures 4.3 et 4.4), documents commerciaux (Figures 4.5 et 4.6), planification des tâches (Figure 4.7), carte géographique des chantiers (Figure 4.8), portail client (Figure 4.9) et module de configuration (Figure 4.10). Cette couverture correspond au périmètre fonctionnel défini au Chapitre 3 et aux processus quotidiens de CIVCO BTP.

Validation et déploiement. La qualité de la solution est assurée par une suite de tests PHPUnit (authentification, CRUD métier, conversion devis → facture, contrôle d'accès par rôle) complétée par une campagne de tests manuels sur les parcours de bout en bout. L'étude comparative d'hébergement a orienté le choix vers **o2switch**, retenu pour sa simplicité de mise en production, son support francophone et son rapport qualité/prix adapté à une PME du secteur BTP.

Bilan. Les principales difficultés — permissions granulaires, impression documentaire, géolocalisation des chantiers et synchronisation frontend / backend — ont été résolues par des services dédiés et une source de vérité unique côté API. L'application est fonctionnelle, testée et prête pour un déploiement chez CIVCO BTP. Le chapitre suivant conclut ce rapport en synthétisant les apports du projet et en esquissant les perspectives d'évolution (notifications temps réel, application mobile, intégration comptable).

Conclusion Générale et Perspectives

Ce projet de fin d'études avait pour objectif de concevoir et développer une application web de gestion et de planification des projets de construction pour **CIVCO BTP**, afin de centraliser les processus métier dispersés entre tableurs, documents papier et échanges informels.

L'application développée au cours de ce stage constitue une réponse concrète et fonctionnelle à cette problématique. Les réalisations techniques couvrent l'intégralité du cycle de vie d'un projet de construction :

- **Gestion de projets** : création et suivi des chantiers par phases, affectation des équipes, géolocalisation sur carte interactive, suivi de l'avancement et des dépenses.
- **Relation client** : fiches clients centralisées, archivage, portail dédié pour consultation et validation des documents.
- **Documents commerciaux** : production de devis, factures et bons de livraison avec lignes de détail, conversion automatisée et suivi des impressions officielles.
- **Suivi contractuel** : gestion des contrats et avenants, compilation depuis des modèles, signature électronique côté client.
- **Planification opérationnelle** : tâches assignées par projet et par collaborateur, tableau de bord avec indicateurs et graphiques.
- **Sécurité et gouvernance** : authentification Sanctum, contrôle d'accès par rôles et permissions (RBAC), journal d'activité pour la traçabilité.
- **Architecture technique** : backend Laravel 13 exposant une API REST, frontend React 19 en SPA, base de données relationnelle, plus de soixante-dix migrations et une suite de tests PHPUnit.

Bilan du projet

Sur le plan fonctionnel, l'application répond aux besoins identifiés lors de la phase d'analyse : chaque service de CIVCO BTP dispose désormais d'un point d'accès unique pour consulter et mettre à jour les informations liées aux chantiers. La centralisation des données élimine les ressaisies manuelles entre tableurs et documents isolés, et le tableau de bord offre une visibilité en temps réel sur l'activité de l'entreprise.

Sur le plan technique, les choix Laravel + React se sont révélés pertinents pour ce type d'application métier : Laravel fournit une base solide pour l'API, la sécurité et la persistance, tandis que React permet de construire une interface réactive et modulaire. L'architecture en trois couches facilite la maintenance et l'ajout de nouveaux modules sans remettre en cause l'existant.

Sur le plan méthodologique, l'approche itérative adoptée pendant le stage — analyse des besoins, conception, développement par modules, tests et validation — a permis de livrer des incréments fonctionnels démontrables à chaque réunion de suivi avec l'encadrant professionnel, M. Mohamed BENABDELLAH-CHAOUNI, et l'encadrant pédagogique, le Pr. Mohammed El Assad.

Apports personnels et compétences acquises

Ce stage au sein de CIVCO BTP m'a permis de consolider et d'approfondir plusieurs compétences techniques et transversales :

- **Développement full-stack** : conception et implémentation d'une application complète, de la base de données à l'interface utilisateur.
- **Architecture logicielle** : application des patrons MVC, services métier, API REST et SPA dans un contexte professionnel réel.
- **Sécurité applicative** : mise en œuvre concrète de l'authentification, du RBAC, de la protection CSRF et de la journalisation.
- **Modélisation** : rédaction de spécifications fonctionnelles, diagrammes UML et modèle de données relationnel.
- **Gestion de projet** : planification, priorisation des fonctionnalités, suivi Kanban via Trello, communication régulière avec les parties prenantes.
- **Rigueur documentaire** : rédaction du présent rapport et documentation technique du code produit.

Au-delà des compétences techniques, ce projet m'a sensibilisé aux réalités du secteur BTP au Maroc : la diversité des intervenants sur un chantier, l'importance de la traçabilité documentaire et les contraintes de terrain qui influencent les choix ergonomiques de l'application.

Limites et axes d'amélioration

Malgré les résultats obtenus, certaines limites subsistent et ouvrent des pistes d'amélioration :

- **Couverture de tests** : la suite PHPUnit couvre les flux critiques mais ne teste pas encore l'ensemble des modules (contrats, portail client). L'ajout de tests end-to-end (Cypress, Playwright) renforcerait la confiance avant un déploiement en production.
- **Performance à l'échelle** : les tests ont été réalisés en environnement local avec un volume de données modéré. Un benchmark sur des milliers de projets et documents serait nécessaire avant un usage intensif.
- **Formation utilisateurs** : l'adoption de l'outil par l'ensemble des services nécessitera un accompagnement au changement et une documentation utilisateur dédiée.
- **Déploiement production** : une étude comparative OVHcloud / o2switch a conduit au choix d'**o2switch** ; la mise en ligne effective reste à finaliser (configuration du domaine, déploiement du build React et exécution des migrations).

Perspectives

L'application offre plusieurs pistes d'évolution à court et moyen terme pour CIVCO BTP.

Évolutions fonctionnelles :

- **Application mobile** : version allégée pour les conducteurs de travaux sur chantier (consultation des tâches, mise à jour d'avancement, photos).
- **Business Intelligence** : tableaux de bord avancés, export Excel/PDF des indicateurs, analyse de la rentabilité par projet.
- **Gestion des stocks** : suivi des matériaux et approvisionnements liés aux chantiers.
- **Notifications** : alertes email ou push pour les échéances de tâches, validations en attente et paiements.
- **Signature avancée** : intégration d'un prestataire de signature électronique qualifiée pour les contrats à forte valeur juridique.

Évolutions techniques :

- **Déploiement production (o2switch)** : publication de l'application sur l'hébergement retenu (build React, configuration Laravel, certificat SSL, sauvegardes automatiques de la base MySQL).

- **Montée en charge** : si le volume d'utilisateurs augmente significativement, migration vers une offre VPS OVHcloud ou cloud managé pour plus de flexibilité.
- **Tests** : extension de la couverture PHPUnit, ajout de tests end-to-end (Cypress ou Playwright) sur les parcours critiques.
- **Performance** : mise en cache des requêtes fréquentes (Redis), pagination optimisée sur les grandes listes.
- **CI/CD** : pipeline d'intégration continue (GitHub Actions) pour exécuter les tests automatiquement à chaque commit.

Ce projet illustre comment une architecture web moderne (Laravel + React) peut transformer la gestion quotidienne d'une entreprise du BTP, en remplaçant des outils dispersés par une plateforme intégrée, sécurisée et adaptée aux profils métier de l'organisation. L'application constitue une base solide sur laquelle CIVCO BTP peut continuer à digitaliser et optimiser ses processus internes, au service de la qualité de ses études et du suivi de ses chantiers.

Bibliographie et Webographie

- [1] R. T. FIELDING, « Architectural Styles and the Design of Network-based Software Architectures », University of California, Irvine, Doctoral dissertation, 2000.
- [2] M. FOWLER, *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.
- [3] LARAVEL LLC, *Laravel Documentation*, 2026. visité le 22 juill. 2026. adresse : <https://laravel.com/docs>
- [4] META PLATFORMS, *React Documentation*, 2026. visité le 22 juill. 2026. adresse : <https://react.dev>
- [5] VITE TEAM, *Vite : Next Generation Frontend Tooling*, 2026. visité le 22 juill. 2026. adresse : <https://vite.dev>
- [6] TAILWIND LABS, *Tailwind CSS Documentation*, 2026. visité le 22 juill. 2026. adresse : <https://tailwindcss.com/docs>
- [7] ORACLE CORPORATION, *MySQL Documentation*, 2026. visité le 22 juill. 2026. adresse : <https://dev.mysql.com/doc/>
- [8] POSTGRESQL GLOBAL DEVELOPMENT GROUP, *PostgreSQL Documentation*, 2026. visité le 22 juill. 2026. adresse : <https://www.postgresql.org/docs/>
- [9] LARAVEL LLC, *Laravel Sanctum*, 2026. visité le 22 juill. 2026. adresse : <https://laravel.com/docs/sanctum>
- [10] PROCORE TECHNOLOGIES, *Procore Construction Management Platform*, 2026. visité le 22 juill. 2026. adresse : <https://www.procore.com>
- [11] AGILE ALLIANCE, *Manifeste pour le développement agile de logiciels*, 2001. visité le 22 juill. 2026. adresse : <https://agilemanifesto.org/iso/fr/manifesto.html>
- [12] OVHcloud, *OVHcloud — Hébergement web, VPS et cloud public*, 2026. visité le 22 juill. 2026. adresse : <https://www.ovhcloud.com/fr/>
- [13] o2switch, *o2switch — Hébergement web illimité en France*, 2026. visité le 22 juill. 2026. adresse : <https://www.o2switch.fr/>
- [14] OWASP FOUNDATION, *OWASP Top Ten Web Application Security Risks*, 2025. visité le 22 juill. 2026. adresse : <https://owasp.org/www-project-top-ten/>
- [15] THE PHP GROUP, *PHP Manual*, 2026. visité le 22 juill. 2026. adresse : <https://www.php.net/manual/en/>
- [16] AXIOS CONTRIBUTORS, *Axios — Promise based HTTP client*, 2026. visité le 22 juill. 2026. adresse : <https://axios-http.com/docs/intro>

- [17] LEAFLET CONTRIBUTORS, *Leaflet – an open-source JavaScript library for mobile-friendly interactive maps*, 2026. visité le 22 juill. 2026. adresse : <https://leafletjs.com/>
- [18] RECHARTS CONTRIBUTORS, *Recharts – a composable charting library built on React components*, 2026. visité le 22 juill. 2026. adresse : <https://recharts.org/>
- [19] SEBASTIAN BERGMANN, *PHPUnit – The PHP Testing Framework*, 2026. visité le 22 juill. 2026. adresse : <https://phpunit.de/>
- [20] PROJECT MANAGEMENT INSTITUTE, *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*, 6^e éd. PMI, 2017.
- [21] FÉDÉRATION FRANÇAISE DU BÂTIMENT, *La transformation numérique dans le BTP*, 2024. visité le 22 juill. 2026. adresse : <https://www.ffbatiment.fr/>
- [22] E. MONK et B. WAGNER, *Concepts in Enterprise Resource Planning*, 4^e éd. Cengage Learning, 2013.
- [23] R. C. MARTIN, *Clean Code : A Handbook of Agile Software Craftsmanship*. Prentice Hall, 2008.
- [24] LARAVEL LLC, *Eloquent ORM – Laravel Documentation*, 2026. visité le 22 juill. 2026. adresse : <https://laravel.com/docs/eloquent>
- [25] REMIX SOFTWARE, *React Router Documentation*, 2026. visité le 22 juill. 2026. adresse : <https://reactrouter.com/>
- [26] JSON.ORG, *Introducing JSON*, 2026. visité le 22 juill. 2026. adresse : <https://www.json.org/json-en.html>

Annexe : Annexes

A.1 Extraits de code source

Cette annexe présente les extraits de code source les plus représentatifs de l'application développée pour CIVCO BTP, sélectionnés pour illustrer les contributions techniques décrites dans ce rapport.

A.1.1 Authentification API (AuthController.php)

Le contrôleur d'authentification gère la connexion des utilisateurs internes via Laravel Sanctum. La méthode login valide les identifiants, régénère la session et retourne le contexte utilisateur (profil, rôles, permissions).

```
1 public function login(LoginRequest $request,
2     AuthContextService $authContext): JsonResponse
3 {
4     $credentials = $request->only('email', 'password');
5
6     if (! Auth::attempt($credentials, $request->boolean('remember'))) {
7         throw ValidationException::withMessages([
8             'email' => [__('auth.failed')],
9         ]);
10    }
11
12    $user = Auth::user();
13
14    if (! $user->canAccessApplication()) {
15        Auth::logout();
16        throw ValidationException::withMessages([
17            'email' => [CheckUserStatus::DEACTIVATED_MESSAGE],
18        ]);
19    }
20
21    if ($request->hasSession()) {
22        $request->session()->regenerate();
23    }
24
```

```

25     return response()->json($authContext->forUser($user));
26 }

```

Listing A.1: AuthController.php : connexion utilisateur (extrait)

A.1.2 Conversion devis vers facture (QuoteService.php)

Le service métier QuoteService encapsule la logique de conversion d'un devis accepté en facture. L'opération s'exécute dans une transaction base de données pour garantir la cohérence des enregistrements.

```

1  public function convertToInvoice(Quote $quote,
2      ?int $dispatchNoteId = null): Invoice
3  {
4      if ($quote->status !== QuoteStatus::Accepted) {
5          throw new InvalidArgumentException(
6              'Only accepted quotes can be converted. ');
7      }
8
9      if ($quote->invoice()->exists()) {
10         throw new InvalidArgumentException(
11             'This quote has already been converted. ');
12     }
13
14     return DB::transaction(function () use ($quote, $dispatchNoteId) {
15         $quote->load('lines');
16
17         $invoice = Invoice::query()->create([
18             'company_id' => $quote->company_id,
19             'client_id'  => $quote->client_id,
20             'project_id' => $quote->project_id,
21             'quote_id'   => $quote->id,
22             'reference'  => $this->invoiceReferenceService
23                 ->nextForCompany($quote->company_id),
24             'status'     => InvoiceStatus::Draft,
25             'total_ht'   => $quote->total_ht,
26             'total_tax'  => $quote->total_tax,
27             'total_ttc'  => $quote->total_ttc,
28         ]);
29
30         foreach ($quote->lines as $index => $line) {
31             $invoice->lines()->create([
32                 'sort_order' => $index + 1,
33                 'description' => $line->description,
34                 'quantity'   => $line->quantity,
35                 'unit_price_ht' => $line->unit_price_ht,
36                 'line_total_ht' => $line->line_total_ht,

```

```

37         'line_total_ttc' => $line->line_total_ttc,
38     ]);
39 }
40
41     return $invoice->load(['client', 'project', 'lines']);
42 });
43 }

```

Listing A.2: QuoteService.php : conversion devis → facture (extrait)

A.1.3 Modèle Project (Project.php)

Le modèle Eloquent Project centralise les relations métier d'un chantier : client, phases, équipe, devis et factures.

```

1 class Project extends Model
2 {
3     protected $fillable = [
4         'company_id', 'client_id', 'reference', 'title',
5         'status', 'budget', 'progress_percent',
6         'start_date', 'end_date', 'latitude', 'longitude',
7     ];
8
9     protected function casts(): array
10    {
11        return [
12            'status' => ProjectStatus::class,
13            'start_date' => 'date',
14            'end_date' => 'date',
15            'budget' => 'decimal:2',
16            'progress_percent' => 'decimal:2',
17        ];
18    }
19
20    public function client(): BelongsTo
21    {
22        return $this->belongsTo(Client::class);
23    }
24
25    public function phases(): HasMany
26    {
27        return $this->hasMany(ProjectPhase::class);
28    }
29
30    public function teamMembers(): BelongsToMany
31    {
32        return $this->belongsToMany(User::class, 'project_user');

```

```

33     }
34
35     public function quotes(): HasMany
36     {
37         return $this->hasMany(Quote::class);
38     }
39 }

```

Listing A.3: Project.php : relations Eloquent (extrait)

A.1.4 Client HTTP frontend (client.js)

Le module `api/client.js` configure l'instance Axios partagée par tous les modules frontend. Il transmet les cookies Sanctum, le jeton CSRF et le contexte société (X-Company-Id).

```

1  const api = axios.create({
2    baseUrl: import.meta.env.VITE_API_URL ?? '',
3    withCredentials: true,
4    headers: {
5      Accept: 'application/json',
6      'Content-Type': 'application/json',
7      'X-Requested-With': 'XMLHttpRequest',
8    },
9  })
10
11 api.interceptors.request.use(async (config) => {
12   if (activeCompanyId) {
13     config.headers['X-Company-Id'] = activeCompanyId
14   }
15
16   const method = config.method?.toLowerCase()
17   if (['post', 'put', 'patch', 'delete'].includes(method)) {
18     let token = getCookie('XSRF-TOKEN')
19     if (!token) {
20       await api.get('/sanctum/csrf-cookie')
21       token = getCookie('XSRF-TOKEN')
22     }
23     if (token) {
24       config.headers['X-XSRF-TOKEN'] = token
25     }
26   }
27
28   return config
29 })
30
31 api.interceptors.response.use(

```

```

32 (response) => response,
33 (error) => {
34     if (error.response?.status === 401) {
35         window.dispatchEvent(new CustomEvent('auth:unauthorized'))
36     }
37     return Promise.reject(error)
38 }
39 )

```

Listing A.4: client.js : configuration Axios et intercepteurs (extrait)

A.2 Configuration : variables d'environnement

Le fichier .env du backend centralise la configuration de l'application. Le Tableau A.1 décrit les variables principales utilisées en développement et en production.

Tableau A.1 : Variables d'environnement principales

Variable	Rôle
APP_URL	URL publique du backend Laravel
DB_CONNECTION	Pilote SGBD (mysql ou pgsql)
DB_DATABASE	Nom de la base de données
SESSION_DRIVER	Stockage des sessions (database recommandé)
SANCTUM_STATEFUL_DOMAINS	Domaines autorisés pour les cookies SPA
FRONTEND_URL	URL du frontend React (CORS et redirections)
MAIL_*	Configuration SMTP pour les notifications

Exemple de configuration minimale pour un environnement de développement local :

```

1 APP_NAME="CIVCO BTP"
2 APP_ENV=local
3 APP_DEBUG=true
4 APP_URL=http://127.0.0.1:8000
5
6 DB_CONNECTION=mysql
7 DB_HOST=127.0.0.1
8 DB_PORT=3306
9 DB_DATABASE=civco_btp
10 DB_USERNAME=root
11 DB_PASSWORD=
12
13 SESSION_DRIVER=database
14 SESSION_LIFETIME=120
15
16 SANCTUM_STATEFUL_DOMAINS=localhost,127.0.0.1:5173

```

```

17
18 FRONTEND_URL=http://127.0.0.1:5173
19
20 MAIL_MAILER=smtp
21 MAIL_HOST=smtp.example.com
22 MAIL_PORT=587
23 MAIL_USERNAME=null
24 MAIL_PASSWORD=null

```

Listing A.5: .env : configuration de développement (extrait)

A.3 Comptes de démonstration

Le seeder de démonstration fournit des comptes utilisateurs permettant de tester les différents profils métier de CIVCO BTP. Le Tableau A.2 liste les identifiants utilisés en environnement de développement.

Tableau A.2 : Comptes de démonstration (mot de passe : password)

Email	Profil	Accès principal
admin@civco.ma	Administrateur	Configuration, tous modules
ingenieur@civco.ma	Chef de projet	Projets, tâches, équipe
compta@civco.ma	Comptable	Factures, paiements
chantier@civco.ma	Chef de chantier	Projets, tâches terrain
tech@civco.ma	Technicien	Tâches assignées, consultation

Ces comptes sont créés par la commande `php artisan migrate --seed` et permettent de valider le contrôle d'accès par rôles décrit au Chapitre 3.

A.4 Référentiel des endpoints API

Le Tableau A.3 présente les principaux endpoints exposés sous le préfixe `/api/v1/`. Toutes les routes (sauf login) sont protégées par le middleware `auth:sanctum` et soumises à un contrôle de permissions.

Tableau A.3 : Principaux endpoints de l'API REST

Méthode	Endpoint	Description
POST	/login	Authentification utilisateur
GET	/me	Profil et permissions de l'utilisateur courant
GET/POST	/projects	Liste et création de projets
GET/PUT	/projects/{id}	Consultation et modification d'un projet
GET/POST	/clients	Liste et création de clients
GET/POST	/quotes	Liste et création de devis
POST	/quotes/{id}/convert	Conversion devis → facture
GET/POST	/invoices	Liste et création de factures
GET/POST	/tasks	Liste et création de tâches
GET/POST	/contracts	Liste et création de contrats
GET	/dashboard	Indicateurs du tableau de bord
GET	/search	Recherche globale (clients, projets, documents)
GET	/activity-logs	Journal des actions (administrateur)

A.5 Schéma des entités principales

Le Tableau A.4 décrit les attributs essentiels des entités métier les plus consultées dans l'application.

Tableau A.4 : Attributs principaux des entités métier

Entité	Attributs clés
Project	reference, title, status, budget, progress_percent, start_date, end_date, latitude, longitude, client_id
Client	name, email, phone, status, is_official, archived_at
Quote	reference, status, total_ht, total_tax, total_ttc, project_id, client_id
Invoice	reference, status, issued_at, due_date, amount_paid, balance_due, quote_id
Task	title, status, priority, due_date, project_phase_id, assigned_to
Contract	reference, status, signed_at, project_id, template_id

Les statuts des entités sont implémentés sous forme d'énumérations PHP (ProjectStatus, QuoteStatus, InvoiceStatus, TaskStatus) garantissant la validité des transitions d'état côté serveur.

A.6 Guide de démarrage rapide

Pour reproduire l’environnement de développement décrit au Chapitre 4, les commandes suivantes permettent d’initialiser l’application à partir du dépôt source :

```
1 # Backend
2 cd backend
3 composer install
4 cp .env.example .env
5 php artisan key:generate
6 php artisan migrate --seed
7 php artisan serve
8
9 # Frontend (dans un second terminal)
10 cd frontend
11 npm install
12 npm run dev
```

Listing A.6: Commandes d’initialisation de l’application

L’application est alors accessible à l’adresse `http://127.0.0.1:5173` (frontend) avec le proxy Vite redirigeant les appels API vers `http://127.0.0.1:8000`. Les comptes de démonstration listés en section 3 permettent de tester les différents profils métier.